



Beyond Kalman filters: deep learning-based filters for improved object tracking

Momir Adžemović¹ · Predrag Tadić² · Andrija Petrović³ · Mladen Nikolić¹

Received: 14 February 2024 / Revised: 21 August 2024 / Accepted: 5 December 2024 / Published online: 13 December 2024
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

Traditional tracking-by-detection systems typically employ Kalman filters (KF) for state estimation. However, the KF requires domain-specific design choices and it is ill-suited to handling non-linear motion patterns. To address these limitations, we propose two innovative data-driven filtering methods. Our first method employs a Bayesian filter with a trainable motion model to predict an object's future location and combines its predictions with observations gained from an object detector to enhance bounding box prediction accuracy. Moreover, it dispenses with most domain-specific design choices characteristic of the KF. The second method, an end-to-end trainable filter, goes a step further by learning to correct detector errors, further minimizing the need for domain expertise. Additionally, we introduce a range of motion model architectures based on recurrent neural networks, neural ordinary differential equations, and conditional neural processes, that are combined with the proposed filtering methods. Our extensive evaluation across multiple datasets demonstrates that our proposed filters outperform the traditional KF in object tracking, especially in the case of non-linear motion patterns—the use case our filters are best suited to. We also conduct noise robustness analysis of our filters with convincing positive results. We further propose a new cost function for associating observations with tracks. Our tracker, which incorporates this new association cost with our proposed filters, outperforms the conventional SORT method and other motion-based trackers in multi-object tracking according to multiple metrics on motion-rich DanceTrack and SportsMOT datasets.

Keywords Filter · Motion model · Deep learning · Multi-object tracking · Neural ordinary differential equations · Conditional neural processes

1 Introduction

Object tracking is a critical component in various applications, such as autonomous vehicle driving [1, 2], robotics [3], and surveillance [4], where it's essential for understanding and predicting the movements and behaviors of objects in real-time. The tracking-by-detection paradigm, which involves associating detected object bounding boxes over successive frames, has been propelled by the rise of fast and accurate deep learning-based object detection models [5–8]. This approach initially detects objects in each video frame and then associates these detected objects across adjacent frames. An essential part of most association methods is the motion-based association, which relies on the bounding box geometric features and motion models predicting the object's movement patterns [9].

Classical algorithms for motion-based association rely on the Kalman filter (KF) [9–12]. The KF is a strong baseline in the case of predominantly linear motion [9, 13], but fails

✉ Momir Adžemović
pd222011@alas.matf.bg.ac.rs
Predrag Tadić
ptadic@etf.bg.ac.rs
Andrija Petrović
andrija.petrovic@fon.bg.ac.rs
Mladen Nikolić
mladen.nikolic@matf.bg.ac.rs

¹ Department of Computer Science, Faculty of Mathematics, University of Belgrade, Belgrade, Serbia

² Department of Signals and Systems, School of Electrical Engineering, University of Belgrade, Belgrade, Serbia

³ Center for Business Decision Making, Faculty of Organizational Sciences, University of Belgrade, Belgrade, Serbia

to handle non-linear motion effectively [14]. Some alternatives [15–17] perform better for non-linear motion modeling, but they cannot directly learn motion patterns from data and do not figure prominently in object tracking. Both the KF and these alternatives require domain knowledge for heuristics design and careful tuning of the hyperparameters.

To address these drawbacks of the existing approaches, we propose two new filtering methods that leverage non-linear, learnable probabilistic motion models. These filters are designed to directly replace the KF in any tracking solution that employs it and they offer a lot of flexibility in the selection of motion model architectures. The first filtering method uses a learned motion model to estimate a distribution over the object's future position. Then, it uses Bayesian inference to combine this prediction with a new detection. Unlike the KF which requires domain-specific heuristics and hyperparameters for initialization, motion model, process noise, and measurement noise, our Bayesian filters only necessitate a heuristic for approximating the measurement (detector) noise. In the second filtering method, a model learns to predict the object's future position and to combine it with the detector measurement in an *end-to-end* fashion. Thus it eliminates the measurement noise hyperparameters which are needed in the first approach. Due to the proposed training methodology, once the filter is trained, the object detector can be replaced without any need for further adjustments or hyperparameter tuning, providing more flexibility in deployment.

As a core component of these filters, we propose deep learning probabilistic models capable of directly learning non-linear motion patterns from data. Moreover, these models can learn dataset-specific characteristics, such as aspect ratios of objects in pedestrian datasets or frequent object positioning and motion patterns. We opt for Neural ODEs [18] and Neural Processes [19, 20] due to their robustness against missing and noisy data [19–23], which is common in tracking domain.

Further, we improve the tracker logic by proposing a modification to the commonly used Intersection Over Union (IoU) association cost [9, 24]. IoU is a good baseline, but it does not consider the object's scale and position which is important during occlusions. Therefore, we combine IoU with the L1 distance between the scale and position of predicted versus detected bounding boxes, lowering the occurrence of ID switches.

In this work, we simplify our tracking algorithm by focusing solely on motion-based association, similar to the SORT method, and avoid using visual features [10]. This approach enables us to better isolate the contributions of the proposed techniques to the final tracker performance and to evaluate them more precisely.

We summarize our contributions as follows:

- We propose a Bayesian filter based on non-linear motion models as an alternative to the standard KF.
- We propose an end-to-end learnable filter that requires no domain-specific choices and can adapt to an arbitrary detector without the need for additional training or hyperparameter adjustments.
- We consider several deep learning-based non-linear motion models designed for trajectory extrapolation with uncertainty estimates that fit the filtering paradigm.
- We propose a simple, motion-based association cost function designed for tracking multiple objects undergoing extensive movement and occlusions. By combining it with some of our proposed filters, we build our MoveSORT tracker.

2 Related work

Tracking-by-detection. The tracking-by-detection paradigm was first introduced in the SORT paper [9]. SORT combines an object detector with the KF for estimating the bounding boxes corresponding to the tracked objects and associates them with detections in the current frame based solely on the Intersection Over Union (IoU) measure of overlap. Matching between the track estimations and the detections is a linear assignment problem, which can be solved using the Hungarian algorithm [25]. A match is only possible if the IoU exceeds a specific threshold, denoted as $\text{IoU}_{\min}$. Finally, the state of the motion model is updated based on these detection matches. If a track is not matched with any detection for T_{lost} consecutive frames, it is discarded.

ByteTrack [24] improved SORT by extending the traditional tracking pipeline by associating both high and low-confidence detections to existing tracks (instead of just high-confidence detections), significantly improving the robustness of the tracker. However, both SORT and ByteTrack use the KF with linear motion model, which makes these methods less adaptable to domains with non-linear motion.

Filtering methodology. Traditional filters based on Bayesian inference are popular due to their effectiveness in various applications. Representatives of these include the Kalman filter (KF) [9, 10], as well as its variants: the extended Kalman filter (EKF) [16], unscented Kalman filter (UKF) [17], and particle filter (PF) [15]. Among these, the KF is often used in multi-object tracking applications and our filtering approach is inspired by its success.

The Bot-Sort [12] adaptive KF is used in state-of-the-art solutions for the MOT17 and MOT20 datasets [13]. They proposed KF with a constant velocity linear motion model

combined with a heuristic for noise approximation proportional to the object's bounding box size.

Generally, KF requires a significant amount of domain knowledge to optimize its performance and guarantee satisfactory results. KalmanNet [26] introduced a recursive, data-driven alternative to the EKF, trained to estimate the Kalman gain using an RNN. This approach reduces the need for some KF hyperparameters while still retaining the recursive Bayesian filter pipeline.

Besides these motion-based methods, there are other lines of work which focus on appearance features, like the ones based on particle filters combined with a correlation filter or deep convolutional features for visual and single object tracking. Such methods are robust against occlusions and sudden changes in the object's appearance [27–30], but are not close to our line of work.

Trackers with non-traditional motion models. Although efficient in multi-object tracking, KF still has an issue with error accumulation during consecutive skipped update steps, often occurring due to occlusion. OC_SORT [31] tracking method addresses KF limitations by proposing an observation-centric approach. Namely, instead of relying solely on the linear state estimation, they deal with occlusions by re-initializing the KF by interpolating missing bounding boxes after object's re-identification, in order to correct the error accumulation of filter parameters. The main motivation for this approach stems from the fact that detectors perform per-frame object detection independently and do not accumulate errors over time.

MotionTrack [32] introduced motion model based on the transformer architecture employing a so-called Dynamic MLP with an advanced attention mechanism. If trained on SportsMOT [33], it shows strong results when evaluated on multiple datasets, especially compared to the KF. However, its performance drops significantly if it is not trained on SportsMOT. Another architecture, also named MotionTrack [34], employs a transformer as well but additionally incorporates an interaction module that considers the interactions between the trajectories of tracked objects. This architecture also compensates for camera motion to reduce noise from camera movement.

We also note the recent work in visual object tracking that leverages specialized transformer architectures to enhance tracking performance. Approaches such as the Spatial-Frequency Transformer (SFTransT) [35], Alternating Time-Space Transformers (ATST) [36], and RPformer [37] have explored new methods for preserving spatial priors, emphasizing high-frequency features, and handling complex tracking scenarios through different transformer models. While these efforts are noteworthy, they represent parallel advancements that differ from our approach.

3 Technical background

In this section, we briefly cover non-standard deep learning architectures on which we base our motion models.

3.1 Neural ordinary differential equations

Neural ordinary differential equations [18] (NODE) are a family of deep neural network models that can be understood as Residual networks (ResNet) [38] with continuous depth. A ResNet transforms a hidden state between discrete-time points t and $t + 1$ according to the following equation:

$$\mathbf{h}_{t+1} - \mathbf{h}_t = f_t(\mathbf{h}_t) \quad (1)$$

which can be seen as a discretization of:

$$\frac{d\mathbf{h}(t)}{dt} = f_t(\mathbf{h}_t). \quad (2)$$

Thus, the hidden state can be modeled using an ODE with the initial condition $\mathbf{h}(0) = \mathbf{x}$, where $\mathbf{x}$ is the input data (e.g., image). The model output is given by the final hidden state $\mathbf{h}(T)$ and can be computed using any black-box differential equation solver (e.g. Euler method). A Neural ODE approximates the function f_t using a neural network, denoted as f , which includes trainable parameters θ :

$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta) \quad (3)$$

The loss function L is applied to the outputs of the ODE solver:

$$\begin{aligned} L(\mathbf{z}(t_1)) &= L\left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), t, \theta) dt\right) \\ &= L(\text{ODESolver}(\mathbf{z}(t_0), f, t_0, t_1, \theta)) \end{aligned} \quad (4)$$

where f is a neural network with parameters θ , $\mathbf{z}(t_0)$ is the initial state at time t_0 , and $\mathbf{z}(t_1)$ is the final state at time t_1 .¹ To train this network, we calculate the gradients with respect to its parameters for every output state $\mathbf{z}(t)$. The main challenge in training this type of neural network lies in performing backpropagation through the ODE solver operations. To handle this issue, the ODE solver is treated as a black box, and gradients are computed using the adjoint sensitivity method [39]. It has been shown that NODE tends to be more robust to noise [23] and missing data [22], which is beneficial in object tracking, where missing and badly aligned detections are common.

¹ The differential equation solver ODESolver usually performs multiple intermediate computation steps between time t_0 and t_1 .

3.2 Conditional neural processes

Conditional neural processes (CNP) [20] are a class of deep learning models that focus on the idea of learning to represent stochastic processes. CNPs operate by conditioning on a context set of observed (input, output) pairs to make predictions about unseen target points. Similar to NODEs, CNPs are well-suited for applications in the tracking-by-detection paradigm, where missing detections frequently occur [19–21]. Given a context set $C = \{(x_i, y_i)\}_{i=1}^N$ representing pair of inputs x_i and corresponding observed outputs y_i , and a target input set $X_T = \{x_j\}_{j=N+1}^M$, the CNP predicts the corresponding target outputs $Y_T = \{y_j\}_{j=N+1}^M$:

$$y_j \sim p(y_j | x_j, C) \quad (5)$$

where y_j is the predicted output for target point x_j . The CNP model aggregates information from the context set to make predictions about each target point.

The CNP architecture is summarized by the following equations:

$$r_i = \text{encode}(x_i, y_i) \quad (6)$$

$$r = \text{aggregate}(\{r_i\}_{i=1}^N) \quad (7)$$

$$y_j = \text{decode}(x_j, r) \quad (8)$$

It involves a neural network that encodes each context point (x_i, y_i) into a latent representation r_i , as shown by Eq. (6). This process is followed by an aggregation step, where these latent representations r_i are combined into a global context representation r , as shown by Eq. (7). The aggregated representation r is then used in conjunction with the observed target input x_j to predict the target output y_j , as shown by Eq. (8).

Attentive conditional neural processes. The Attentive Conditional Neural Process [40] (ACNP) is summarized by the following equations:

$$R_C = \text{MHSA}(X_C, Y_C) \quad (9)$$

$$r_j = \text{MHCA}(x_j, X_C, R_C) \quad (10)$$

$$y_j = \text{decode}(x_j, r_j) \quad (11)$$

which we explain further. ACNP enhances the standard Conditional Neural Process (CNP) by incorporating the Multi-Head Self Attention (MHSA) [41] when computing representations as shown by Eq. (9). Then, according to Eq. (10), instead of using simple mean aggregation, the ACNP employs a cross-attention mechanism denoted

as Multi-Head Cross Attention (MHCA) [41], which is designed to aggregate only the relevant context information for each target input. Within this cross-attention function, the target input x_j serves as the query, the context inputs $X_C = \{x_i\}_{i=1}^N$ act as keys, and the context representations $R_C = \{r_i\}_{i=1}^N$ function as values. In simpler terms, instead of the standard mean aggregation, the weighted mean depends on both the context and the target input. The decoding step is similar to the original CNP, but each target uses its own aggregated representation r_j instead of the global one r used in CNP, as shown by Eq. (11).

4 Methodology

In this section we present the main methodological contributions of our work focusing on the deep learning-based filters. Since they rely on deep learning-based motion models, we describe them first and then proceed to build the filters atop of them. Finally, we propose a new method to improve track association.

4.1 Uncertainty-aware object trajectory prediction

First, we define the trajectory prediction task. Assume that a trajectory is observed at non-equidistant time points $T_O = \{t_1, t_2, t_3, \dots, t_n\}$ with corresponding observation vector values $X_O = \{x_1, x_2, x_3, \dots, x_n\}$. Each observation x_i is given by a vector of four bounding box coordinates (top, left, width, height). The task is to estimate the unobserved target values $X_T = \{x_{n+1}, x_{n+2}, x_{n+3}, \dots, x_{n+m}\}$ at target time points $T_T = \{t_{n+1}, t_{n+2}, \dots, t_{n+m}\}$. More concretely, we wish to model the conditional distribution of target values given the observed time points, observation values, and target time points:

$$X_T \sim p(X_T | T_O, X_O, T_T) \quad (12)$$

The motion models we propose are specifically designed for the task of trajectory prediction in object tracking. They are specialized to predict dynamic movements of objects, which is crucial for accurate object tracking.

We choose to model the conditional distribution $p(X_T | T_O, X_O, T_T)$ as a Gaussian distribution, where we assume conditional independence among all target variables. For simplicity, we also assume independence among the target bounding box coordinates, which allows us to model each target variable distribution with a mean vector and a diagonal covariance matrix. Put simply, each motion model produces eight values: the mean value and variance for each of the four coordinates. The assumption of independence among the target bounding box coordinates is

commonly applied in trackers that employ the KF [9, 10, 12]. This not only simplifies the model but also enables more efficient computation of the loss with improved numerical stability [42, 43].

Training. We train our models to minimize the Gaussian negative log-likelihood (NLL) of each target point. We use ground truth annotations for training. The loss is defined as follows:

$$L_{nll}(\mu_{i,j}, \hat{x}_{i,j}, \hat{\sigma}_{i,j}) = \frac{1}{2} \frac{(\mu_{i,j} - \hat{x}_{i,j})^2}{\hat{\sigma}_{i,j}^2} + \frac{1}{2} \log(2\pi\hat{\sigma}_{i,j}^2) \quad (13)$$

where $\mu_{i,j}$ represents the j th ($j = 1, \dots, 4$) target bounding box coordinate for time point t_i , $\hat{x}_{i,j}$ is the corresponding estimated mean, and $\hat{\sigma}_{i,j}^2$ is the corresponding estimated variance (j th element on the diagonal of the diagonal covariance matrix $\hat{\Sigma}_i$). The final loss is calculated by averaging across all coordinates and time points. To ensure computational stability during training, we compute the loss by imposing a small positive lower bound on $\hat{\sigma}_{i,j}$, thereby stopping it from becoming too close to zero [42, 43].

4.2 Motion model architectures

We focus on three model architectures: two from the CNP family (RNN-CNP and ACNP) and one from the NODE family (RNN-ODE). During inference, each model uses a measurement buffer, which will be explained at the end of this section. Unless specified otherwise, all linear network layers are followed by layer normalization [44], and then by leaky ReLU [45].

Instead of working with sequences of raw bounding box coordinates and time values directly, we preprocess them to extract features related to the object's motion. We define the model input features as a sequence of $t_i \parallel x_i$, where x_i is the transformed bounding box at the i th trajectory step, t_i is the transformed time at the i th trajectory step, and $\parallel$ is the concatenation operator. A detailed explanation of these transformations and their effects on the motion model performance is provided in "Appendix [Bounding box and temporal features](#)".

AR-RNN. As a deep motion model baseline, we consider a simple auto-regressive RNN that uses GRU [46] as the recurrent unit. The GRU unit summarizes the trajectory features into a vector representation which is used to estimate the mean and variance of the object's position for the next step.

RNN-ODE. In the RNN-ODE architecture, a GRU unit within the encoder is used to summarize the observed object trajectory (T_O, X_O) into a latent trajectory

representation z_n . This representation is then extrapolated in latent space using an ODE solver, deriving latent representations $z_{n+1}, z_{n+2}, \dots, z_{n+m}$ for the target time points $t_{n+1}, t_{n+2}, \dots, t_{n+m}$. Finally, these latent representations are used to obtain prediction for each point. The resulting RNN-ODE architecture is summarized by the following equations:

$$z_{i+1} = \text{GRU}(z_i, t_i \parallel x_i), \quad i = 0, \dots, n-1 \quad (14)$$

$$z_{i+1} = \text{ODESolver}(z_i, \text{MLP}_{ode}, t_i, t_{i+1}), \quad i = n, \dots, n+m-1 \quad (15)$$

$$\hat{\mu}_i, \hat{\Sigma}_i = \text{MLP}(z_i) \quad (16)$$

RNN-CNP. We apply a simple but effective modification to the original CNP [20]. The resulting RNN-CNP architecture is summarized by the following equations:

$$r_i = \text{MLP}_{enc}(t_i \parallel x_i) \quad (17)$$

$$r = \text{GRU}(\{r_i\}_{i=1}^N) \quad (18)$$

$$\hat{\mu}_j, \hat{\Sigma}_j = \text{MLP}_{dec}(t_j \parallel r) \quad (19)$$

The main modification is replacing the mean aggregation with a GRU unit, which is more expressive and more suitable for sequential data. We use the time points t_i combined with the bounding box features x_i for context, as can be seen in Eq. (17). For the *encode* function, we use an MLP, and for *aggregate*, a GRU unit is used to obtain the context representation r , as shown in Eq. (18). We treat the prediction time points t_j as independent queries. For the *decode* step, the model estimates the target point given the query time point, based on the context r , as shown in Eq. (19). This approach offers two key benefits during decoding: it is fully parallelizable, allowing simultaneous prediction for all *future* time points, and the inference time is not dependent on the temporal distance from the last observed point, a notable advantage over RNN and Neural ODE based variants.

ACNP. We adapt the original ACNP [40] for the trajectory prediction task by using time points as inputs and bounding box features as outputs, similar to the approach in RNN-CNP. We employ the same *decode* operation as used in the RNN-CNP model, as shown in the following equations:

$$R_C = \text{MHSA}(T_O, X_O) \quad (20)$$

$$\mathbf{r}_j = \text{MHCA}(\mathbf{x}_j, \mathbf{X}_O, \mathbf{R}_C) \quad (21)$$

$$\hat{\boldsymbol{\mu}}_j, \hat{\boldsymbol{\Sigma}}_j = \text{MLP}(t_j \parallel \mathbf{r}_j) \quad (22)$$

Motion Model Measurements Buffer. Having access to a detailed history of the object's movement is crucial for the motion model. This is achieved using the *filter buffer*, defined by two key parameters: *default_buffer_size* and *min_buffer_size*. In case there are at least *min_buffer_size* observations in the interval $[t - \text{default_buffer_size}, t - 1]$, the observations before $t_n - \text{default_buffer_size}$ are ignored. Otherwise, this interval is extended so that there are always at least *min_buffer_size* entries in the buffer. This interval extension mitigates the issue of a lack of past observation during occlusions, which generally reduces the motion model's accuracy.

The *default_buffer_size* parameter is always aligned with the motion model's history length used during training. For linear motion, the value for *min_buffer_size* can safely be set to be equal to *default_buffer_size*, but for non-linear motion, where dynamics change rapidly, *min_buffer_size* value should be lower.

4.3 Bayesian filtering with learnable motion model

We propose a method that extends a non-linear probabilistic motion model in a manner analogous to the linear KF algorithm. The motion model's output serves as the Bayesian prior is combined with the likelihood of the measurements to obtain the posterior estimation. We refer to this extension as a deep Gaussian filter, as it performs filtering with the aid of deep motion models.

We cannot directly substitute a deep non-linear motion model into the KF—a recursive Bayesian filter—because applying a non-linear transformation to a

Gaussian-distributed random variable does not typically result in a Gaussian distribution. To address this issue, we make a prediction from raw buffered measurements at each step, as shown in Fig. 1. Since we lack detector measurement uncertainty, we use the same heuristic for the calculation of the measurement noise covariance matrix $\mathbf{R}_n$ as in the Bot-Sort [12], where the variance is proportional to the object's bounding box scale.

Our probabilistic motion models can directly estimate the prior covariance matrix $\hat{\boldsymbol{\Sigma}}_{n+1}$, thereby implicitly learning to estimate the process noise. As a result, only the matrix $\mathbf{R}_{n+1}$ needs to be tuned. We consider the motion model's prediction as a prior distribution $\mathcal{N}(\hat{\boldsymbol{\mu}}_{n+1}, \hat{\boldsymbol{\Sigma}}_{n+1})$ for step $n + 1$, with the detection $\mathbf{x}_{n+1}$ and the measurement noise covariance matrix $\mathbf{R}_{n+1}$ forming the measurement likelihood distribution $\mathcal{N}(\mathbf{x}_{n+1}, \mathbf{R}_{n+1})$. By applying Bayes' theorem to combine information from two Gaussian distributions [47], we derive the posterior distribution $\mathcal{N}(\tilde{\boldsymbol{\mu}}_{n+1}, \tilde{\boldsymbol{\Sigma}}_{n+1})$, as follows:

$$\tilde{\boldsymbol{\Sigma}}_{n+1} = (\hat{\boldsymbol{\Sigma}}_{n+1}^{-1} + \mathbf{R}_{n+1}^{-1})^{-1} \quad (23)$$

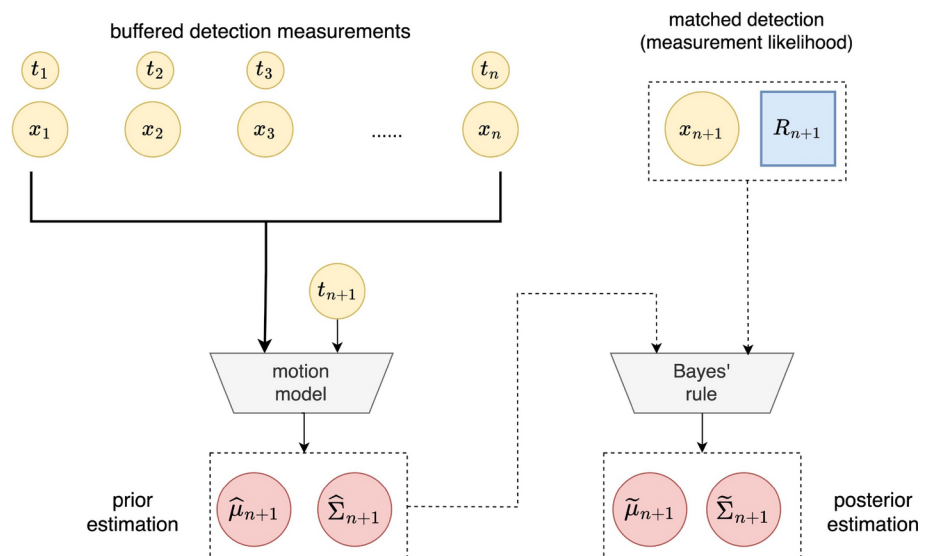
$$\tilde{\boldsymbol{\mu}}_{n+1} = \tilde{\boldsymbol{\Sigma}}_{n+1} \hat{\boldsymbol{\Sigma}}_{n+1}^{-1} \hat{\boldsymbol{\mu}}_{n+1} + \tilde{\boldsymbol{\Sigma}}_{n+1} \mathbf{R}_{n+1}^{-1} \mathbf{x}_{n+1} \quad (24)$$

This methodology is visually illustrated in Fig. 1. In cases where a measurement is missing, obtaining a posterior estimation is not possible; therefore, we use the prior as a substitute.

4.4 End-to-end learnable filtering

When both the prior distribution and the measurement likelihood are precisely known, Bayes' rule offers a definitive

Fig. 1 Visualization of a Bayesian filter incorporating non-linear, deep learning-based motion models. The measurement buffer comprises detection measurements $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ at times $\{t_1, t_2, \dots, t_n\}$, representing the object's trajectory history. These measurements, along with the target time t_{n+1} serve as input to the motion model. The motion model predicts a prior normal distribution $\mathcal{N}(\hat{\boldsymbol{\mu}}_{n+1}, \hat{\boldsymbol{\Sigma}}_{n+1})$, which is used by the tracker in the association step. The matched detection $\mathbf{x}_{n+1}$, accompanied by a heuristic-based measurement noise matrix $\mathbf{R}_{n+1}$, represents the measurement likelihood $\mathcal{N}(\mathbf{x}_{n+1}, \mathbf{R}_{n+1})$. Finally, Bayes' rule is applied to derive the posterior estimation $\mathcal{N}(\tilde{\boldsymbol{\mu}}_{n+1}, \tilde{\boldsymbol{\Sigma}}_{n+1})$ given the prediction prior and measurement likelihood



method for determining the posterior distribution. However, the measurement likelihood is often not known a priori and is approximated using a heuristic based on the domain knowledge. As an alternative to such heuristic, we propose a method that directly learns to filter the measurement noise.

Our approach consists of two stages. In the first stage, we estimate the prior in a manner akin to conventional motion models. Then, instead of employing Bayes' theorem with a heuristically chosen covariance matrix R_n , we merge the prior information with the newly observed measurement value by feeding them into a neural network to derive the posterior estimation. All parts of this pipeline are trained together end-to-end. Hence, the name we chose for this approach. This method eliminates the need for manually tuned uncertainty hyperparameters, like the covariance matrices for process and measurement noise in the KF. For the neural network tasked with posterior estimation, we use a GRU unit aiming to learn filtering based on the measurement history. This filtering process is visualized in Fig. 2.

Training. We augment the loss by a term corresponding to the estimation of the posterior and train with ground truth annotations $\mu_{i,j}$ as targets. The new loss function is as follows:

$$\begin{aligned} L_{e2e}(\mu_{i,j}, \hat{\mu}_{i,j}, \hat{\sigma}_{i,j}, \tilde{\mu}_{i,j}, \tilde{\sigma}_{i,j}) \\ = L_{nll}(\mu_{i,j}, \hat{\mu}_{i,j}, \hat{\sigma}_{i,j}) \\ + L_{nll}(\mu_{i,j}, \tilde{\mu}_{i,j}, \tilde{\sigma}_{i,j}) \end{aligned} \quad (25)$$

where L_{nll} is defined as in Eq. (13). In real-world applications, the bounding boxes are produced by non-ideal detectors, and the measurements x can be seen as noisy versions of ground-truth bounding boxes μ . We define a denoising task by adding Gaussian noise of varying variance to μ to synthesize the inputs x to our model and train the model to reconstruct μ . Thus, after training, no further

hyperparameter tuning for a specific detector is required, unlike the traditional Bayesian filter. Therefore, any detector can be used with thus trained filter.

We propose two specific end-to-end filtering architectures—NODEFilter and RNNFilter.

NODEFilter. NODEFilter architecture combines NODE and GRU networks in an end-to-end fashion, resulting in high robustness at the cost of slow inference speed [22, 23]. As seen in Fig. 3, the hidden state is extrapolated by the NODE between two observations, yielding the prediction, while the GRU is used to incorporate the information from the noisy measurement and produce the final posterior estimation.

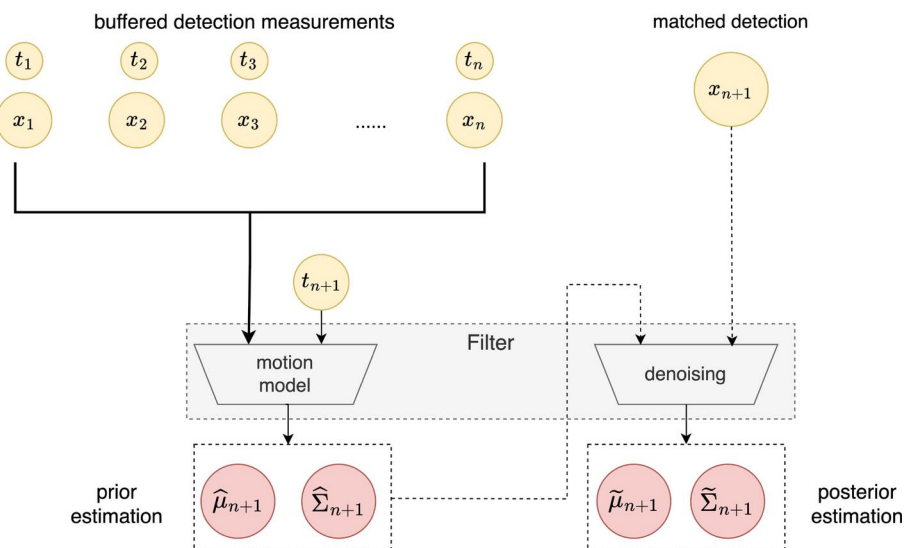
NODEFilter's state is defined through a latent vector representation, summarizing the trajectory history. The model updates the latent state with intermittent *predict* and *update* operations. We initialize the latent vector $\tilde{z}_0$ to a default value (e.g., all zeros) representing an empty trajectory. The result from the *predict* operation $\tilde{z}_{i+1}$ is obtained through extrapolation from the previous state using the Neural ODE with a MLP_{ode} function, as shown in Eq. (26). The predicted latent state $\tilde{z}_{i+1}$ is combined with the new measurement x_{i+1} by the GRU neural network, resulting in the *update* $\tilde{z}_{i+1}$, as shown in Eq. (28). Note that the latent variable is treated as the hidden state of the GRU network. The predictions are acquired by projecting from the latent space to the measurement space, as shown in Eqs. (27) and (29). NODEFilter operations are as follows:

$$\hat{z}_{i+1} = \text{ODESolver}(\tilde{z}_i, MLP_{ode}, t_i, t_{i+1}) \quad (26)$$

$$\hat{\mu}_{i+1}, \hat{\Sigma}_{i+1} = MLP_{predict}(\hat{z}_{i+1}) \quad (27)$$

$$\tilde{z}_{i+1} = \text{GRU}(x_{i+1} \parallel t_{i+1}, \hat{z}_{i+1}) \quad (28)$$

Fig. 2 Overview of the two-stage end-to-end filtering process. The differences with respect to Fig. 1 are as follows. First, there is no requirement for providing a manually tuned measurement noise matrix R_{n+1} . Second, a neural network implicitly learns to filter out measurement noise, replacing the application of Bayes' rule



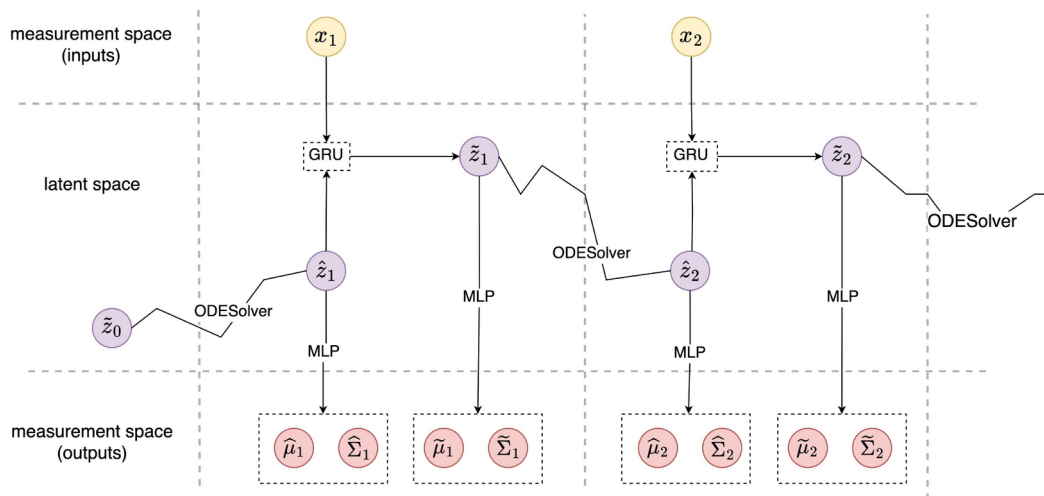


Fig. 3 Visualization of the NODEFilter architecture. The latent state z_n summarizes the trajectory up to time t_n . Initially, z_0 , a zero vector, represents an empty trajectory. The ODESolver is employed to extrapolate-or *predict*-the latent trajectory from z_0 to z_1 . Subsequently, the mean $\hat{\mu}_1$ and covariance $\hat{\Sigma}_1$ are derived from the latent

$$\tilde{\mu}_{i+1}, \tilde{\Sigma}_{i+1} = \text{MLP}_{\text{update}}(\tilde{z}_{i+1}) \quad (29)$$

If a measurement is missing, the GRU update to the latent state is skipped, and the representation from the prediction step $\hat{z}_{i+1}$ is used instead of $\tilde{z}_{i+1}$ to obtain the posterior estimation.

Our proposed NODEFilter architecture closely resembles the ODE-RNN encoder [22] and GRU-ODE-Bayes [48], alternating between using NODE for extrapolation between observations and GRU for observation noise filtering. In contrast, in our architecture, the NODE and GRU components are explicitly specialized for distinct sub-tasks within the architecture. While GRU is our default choice for the filtering component, our architecture allows for a flexible selection of that component.

Our end-to-end filters can be used either as a recursive filter or in combination with a measurement buffer. Through our experiments, we have observed that the latter option tends to be more robust, primarily due to reduced error accumulation, and offers greater flexibility in handling trajectory input features.

RNNFilter. To demonstrate the generality of our framework, we also consider the RNNFilter architecture. By substituting the continuous ODESolver operation with a GRU layer in Eq. (26), we develop the RNNFilter. Namely, our prediction step is as follows:

$$\hat{z}_{i+1} = \text{GRU}(t_{i+1}, \tilde{z}_i) \quad (30)$$

We have deferred the exploration of filters based on neural processes to future work.

representation $\hat{z}_1$. Upon observing the first measurement x_1 , a GRU network produces the *updated* latent state $\tilde{z}_1$ by filtering out the measurement noise, resulting in final estimates of $\hat{\mu}_1$ and $\hat{\Sigma}_1$. This procedure is iteratively repeated. In scenarios where a measurement is missing, the *update* step involving the GRU is omitted

4.5 MoveSORT

Our filters can serve as a direct replacement for the KF in any framework relying on the tracking-by-detection paradigm. We adopt the popular SORT [9] method as a baseline, and introduce an improved association method, which we explain here. We refer to the integration of SORT with our custom filter and the new association method as MoveSORT.

Hybrid Association Method. The standard negative IoU with gating² [12, 24, 31, 32, 49] does not discriminate between objects of different scales and positions. This results in a cost matrix for linear assignment based predominantly on overlap, thereby failing to adequately account for positional and scale discrepancies between the predicted and detected bounding boxes. These differences are especially important during instances of crowd occlusion, where the precision of detectors diminishes. The importance of positional and scale cues is further noticeable in scenarios involving a horizontal camera view, attributable to the perspective projection effects. For instance, the bounding box representing an individual situated further from the camera will manifest as smaller in terms of width and height compared to that of an individual located nearer. Moreover, the bounding box of the more distant individual will also be positioned upper on the projected image, even though both subjects are on the same ground level within the scene. To exploit these cues, our hybrid association method augments the conventional negative IoU technique by integrating a

² A match between a prior and detection is not possible if the minimum IoU constraint is not satisfied.

weighted Manhattan distance between the predicted bounding box prior $\hat{\mu}_{n+1}$ and a detection D .

$$C_{move}(\hat{\mu}_{n+1}, D) = C_{-iou}(\hat{\mu}_{n+1}, D) + \lambda \cdot \|\hat{\mu}_{n+1} - D\|_1 \quad (31)$$

This proposed association method also considers positional offsets and scale. This makes association easier in crowded scenarios, where bounding boxes tend to overlap. This should lead to a lower rate of ID switches.

Although we could account for uncertainty in motion model prediction during association by using the Mahalanobis distance, which calculates the distance from the detector measurement to the motion model's estimated distribution characterized by mean $\hat{x}_i$ and covariance $\hat{\Sigma}_i$, we adhere to the standard practice of calculating distances between two bounding boxes as points, as demonstrated in [12, 24, 31, 49, 50].

5 Experimental evaluation

We evaluate the proposed methods using four MOT datasets—DanceTrack [14], SportsMOT [33], MOT17 and MOT20 [13], and one SOT dataset—LaSOT [51, 52]. Our interest lies in assessing the effectiveness of the proposed filters in motion-based association for Multi-Object Tracking (MOT), their accuracy in the context of Single Object Tracking (SOT), and their robustness against detector noise. We stress that we do not consider MOT17 and MOT20 proper benchmarks for our methods since they exhibit linear motion and our methods are specifically designed for non-linear motion. However, they are standard MOT benchmarks and we include them in the evaluation for transparency.

5.1 Experimental setup

Here, we discuss the design choices and hyperparameters of all components of our tracker, including the detector, tracker logic, and filters.

Detector. We use the YOLOX detector [5] for all MOT datasets, with the detection confidence threshold set to 0.6. For LaSOT, we train YOLOv8 [6]. Detailed information about each detector can be found in “Appendix 2”.

Tracker logic. We have determined that the choice of $\lambda = 5$ in Eq. (31) for our hybrid association method is efficient across all datasets, ensuring that the IoU cost remains dominant, but still reducing the rate of ID switches.

A track is deleted if it is lost, i.e. not matched with any observations, for T_{lost} consecutive frames. We tuned T_{lost} separately for each dataset, choosing values of 30, 30, 20,

and 60 for DanceTrack, SportsMOT, MOT17, and MOT20, respectively.

Once an object is re-identified, linear interpolation is performed between its current bounding box position and the last known position before the track was lost, thereby filling gaps in the object track. This method is a standard approach, often used for MOT tracking [12, 24, 31, 34, 53]. For DanceTrack, linear interpolation proved to be nonbeneficial if an object is lost for more than 5 frames. Alternative interpolation methods can be investigated in future work. A new track is initialized if an object is successfully matched for 3 consecutive frames; otherwise, it is deleted. This approach effectively reduces the number of false positives. We set the IoU threshold IoU_{min} to 0.25 unless specified otherwise.

Filters. The filter's *default_buffer_size* is always set to the length of the trajectory history used during model training. The same applies to the *min_buffer_size* unless specified otherwise. Additional augmentations include simulated missing detections and bounding box localization errors, resulting in improved robustness to the detector errors. These augmentations are explained in detail in “Appendix Augmentations”.

Trajectory features, which are explained in more detail in “Appendix Bounding box and temporal features”, are pre-processed using *standardized scaled first-order difference* for MOT17 and MOT20 datasets, and *standardized relative to last observation* for DanceTrack, SportsMOT and LaSOT datasets. For all our Neural ODE-based architectures, we use the Runge–Kutta fourth-order method [54] as the ODE solver, with a fixed step size of 0.25, as the default setup for both inference and training.

We use the Bot-SORT implementation of the KF as a baseline for performance comparison for all datasets.

Software and hardware. All training and evaluation experiments were conducted on a Linux machine with Ubuntu 22.04 distribution equipped with an Intel i7-12700K processor, 32GB of RAM, and a single NVIDIA GeForce RTX 3070 GPU with 8GB of VRAM. However, we note that all proposed motion models are lightweight in terms of GPU memory and can be trained and evaluated on GPUs with as little as 1GB of VRAM. All models are trained and evaluated using the PyTorch framework [42].

5.2 Evaluation metrics

In evaluating the effectiveness of our approach for multi-object tracking, we have used the higher order metrics (HOTA, AssA, DetA), IDF1, and MOTA—Multi-Object Tracking Accuracy [55]. MOTA, determined by FN, FP, and IDs, is particularly sensitive to the detectors' performance. IDF1 indicates the quality of associations [55]. We use these

metrics for DanceTrack, SportsMOT, MOT17, and MOT20 datasets.

Moreover, we analyze the filters on the task of single-object tracking, where accuracy is measured by the average IoU between the ground truth and our prediction. We perform evaluation of both filters' prior and posterior accuracies.

5.3 Tracker evaluation on DanceTrack

DanceTrack consists of 100 videos, specifically annotated for multi-object tracking of dancers. Tracking dancers, characterized by similar appearances, frequent occlusions, and fast non-linear movements, poses a significant challenge. The dataset is divided into 40 training, 25 validation, and 35 test videos [14].

Table 1 provides a comparison of various tracker methods that use the same detector.³ To ensure a fair comparison with the meaningful baselines, all modifications that are not related to the filters or the association cost heuristic are also

Table 1 Evaluation results for various tracking methods on DanceTrack test set

Method	HOTA	DetA	AssA	MOTA	IDF1
<i>Motion and appearance-based</i>					
DeepSORT [10]	45.6	71.0	29.7	87.8	47.9
Deep OC_SORT [53]	61.3	81.6	45.8	91.8	61.5
<i>Motion-based</i>					
SORT [9]	47.9	72.0	31.2	91.8	50.8
SORT-tuned	51.2	80.5	32.6	91.5	49.7
ByteTrack [24]	47.3	71.6	31.4	89.5	52.5
BoT-SORT [12]	53.6	79.9	36.1	92.3	55.3
OC_SORT [31]	55.1	80.4	38.0	89.4	54.9
MotionTrack (DanceTrack) [32]	52.9	80.9	34.7	91.3	53.8
SparseTrack [49]	55.5	78.9	39.1	91.3	58.3
<i>Motion-based</i>					
MoveSORT-KF (ours)	53.3	80.7	35.3	<i>91.7</i>	51.2
MoveSORT-AR-RNN (ours)	54.0	80.5	36.3	<i>91.7</i>	54.7
MoveSORT-ACNP (ours)	54.3	80.2	36.8	91.3	54.4
MoveSORT-RNN-CNP (ours)	55.9	80.6	38.8	91.8	56.5
MoveSORT-RNN-ODE (ours)	55.6	80.7	38.5	91.8	56.3
MoveSORT-RNNFilter (ours)	<i>56.0</i>	<i>81.4</i>	38.7	91.4	56.0
MoveSORT-NODEFilter (ours)	56.1	81.6	38.7	91.8	56.0

The best results for motion-based methods are in bold for each metric, while the second-place results are italicized. Results for trackers that use appearance-based association are included for reference only, as they are not comparable to the other methods. The BoT-SORT results are produced by us

³ Detailed information about DanceTrack's object detector setup can be found in "Appendix DanceTrack".

applied to the SORT baseline. Thus, all achieved improvements compared to the baseline are strictly the result of applying our methodology and not of using a weak baseline. Our SORT baseline, SORT-Tuned, has increased the HOTA by 3.3% compared to the DanceTrack SORT tracker [14], as can be seen from Table 1.

The only difference between MoveSORT-KF and SORT-Tuned is in the association cost heuristic. MoveSORT-KF uses our proposed hybrid loss, instead of the negative IoU cost used in SORT-Tuned. This results in an improvement of 2.1% for MoveSORT-KF compared to the SORT-Tuned tracker, demonstrating the effectiveness of this simple heuristic.

Substituting the KF with any deep learning-based filter in the MoveSORT framework leads to further improvements, as can be seen in Table 1. AR-RNN proves to be a stronger baseline than KF on this dataset, leading by 0.7% in HOTA. All our filters, ACNP, RNN-CNP, RNN-ODE, RNNFilter, and NODEFilter, significantly outperform the KF baseline, with NODEFilter showing the best performance, achieving a total improvement of 2.8% in HOTA compared to KF, and a 2.1% improvement in HOTA compared to AR-RNN. It is noteworthy that, compared to Bayesian filters, NODEFilter excels in the DetA metric, as it performs filtering more efficiently. Additionally, replacing the GRU (MoveSORT-RNNFilter) with NODE (MoveSORT-NODEFilter) in the filter extrapolation step results in a 0.3% improvement in DetA performance and 0.4% in MOTA. MoveSORT-NODEFilter outperforms SORT-tuned baseline by 4.9%, and DanceTrack SORT baseline by 8.2%.

We also compared our best tracker, MoveSORT-NODEFilter, to other motion-based trackers in the literature on the DanceTrack test set, while methods that employ appearance-based association are included just for reference. Our tracker outperforms the transformer-based motion model, specifically MotionTrack [32], by 3.2% HOTA when it is trained only on the DanceTrack training dataset. MotionTrack performs stronger when trained on the SportsMOT dataset, which offers more diverse motion. However, it's important to note that we used only DanceTrack for model training. We also outperform OC_SORT by 1.0% in HOTA. The strongest motion-based method after ours is SparseTrack [49], which is based on a depth estimation heuristic, but our method still surpasses SparseTrack by 0.6% in terms of HOTA.

All our methods use the same post-processing algorithm,⁴ which yields a consistent improvement of 0.2% in terms of the HOTA metric. Specifically, for this dataset, we use an IoU_{min} threshold of 0.2.

⁴ Post-processing algorithm includes linear interpolation up to 5 frames, removal of all tracks that are shorter than 20 frames, and inclusion of track bounding boxes during the 2-frame initialization period.

Table 2 Comparison of SORT tracker performance using different parameters on the DanceTrack validation set

Row	T_{lost}	Association method	Filter	IoU_{min}	Post-process	HOTA	DetA	AssA	MOTA	IDF1
1	1	IoU	KF	0.3	No	46.3	78.5	27.5	88.0	40.7
2	30	IoU	KF	0.3	No	49.5 (+3.2)	77.8 (−0.7)	31.6 (+4.2)	88.9 (+0.9)	48.4 (+7.7)
3	30	IoU and L1	KF	0.3	No	51.8 (+5.5)	77.7 (−0.8)	34.7 (+7.2)	89.0 (+1.0)	51.2 (+10.5)
4	30	IoU and L1	NF	0.3	No	54.1 (+7.8)	78.5 (+0.0)	37.5 (+10.0)	88.9 (+0.9)	52.9 (+12.2)
5	30	IoU and L1	NF	0.2	No	55.2 (+8.9)	78.1 (−0.4)	39.3 (+11.8)	88.9 (+0.9)	55.1 (+14.4)
6	30	IoU and L1	NF	0.2	Yes	55.4 (+9.1)	78.4 (−0.1)	39.3 (+11.8)	89.7 (+1.7)	55.2 (+14.5)
7	30	IoU	NF	0.2	No	51.0	78.0	33.5	88.8	50.1

We use acronym NF for NODEFilter architecture. Parentheses include the total improvement compared to the SORT baseline. Bold indicate modifications made to the tracker compared to the previous row. The total improvement compared to SORT includes 9.1% in terms of HOTA, 1.7% in terms of MOTA, and 14.5% in terms of IDF1 metrics

Table 3 Tracker performance comparison during occlusions on DanceTrack validation set

filter	Precision	Recall	f1-score
KF	64.4	80.4	71.5
AR-RNN	63.7	83.1	72.1
ACNP	64.6	83.7	72.9
RNN-CNP	63.6	83.8	72.3
RNNODE	64.4	83.8	72.9
RNNFilter	64.0	83.2	72.4
NODEFilter	64.5	84.1	73.0

Numbers are shown as percentages. The best results are in bold for each metric

Ablation study. We conducted an incremental ablation study to analyze the contribution of each modification from the SORT baseline to MoveSORT-NODEFilter. The results are detailed in Table 2. We can observe that our proposed hybrid association method improves SORT tracker performance by 2.3%. This method is even more effective when combined with a more accurate filter which gives an improvement of 4.2% in HOTA (comparison between 5th and 7th row in the table). By replacing KF with a stronger filter in SORT without changing the association method improves tracker performance by 1.5% in HOTA (comparison between 2nd and 7th row in the table).

Occlusion handling analysis. We analyzed tracker performance during occlusions to better understand the benefits of the exact motion model. Specifically, we define a true positive (TP) occlusion match when an object is correctly associated after an occlusion, a false positive (FP) when an ID switch occurs due to incorrect association after an occlusion, and a false negative (FN) when an object is not found after an occlusion. We consider an object occluded if it is not associated for at least five frames. Using the TP, FP, and FN values, we calculate standard classification metrics: precision, recall, and F1-score. The analysis results are presented in Table 3, from which we observe that the key difference

between the KF and our proposed motion models lies in their ability to find objects after occlusion while maintaining the same precision by not increasing the number of ID switches. Specifically, our NODEFilter method outperforms KF by 3.7% in terms of recall and 1.5% in terms of f1-score.

5.4 Tracker evaluation on SportsMOT

SportsMOT is a large-scale multi-object tracking dataset comprising 240 video sequences with over 150,000 frames. The dataset includes three sports categories: basketball, volleyball, and football. Characterized by fast and variable-speed motion, as well as similar object appearances, SportsMOT presents a significant challenge for object tracking. The dataset is divided into training, validation, and test sets, containing 45, 45, and 150 video sequences, respectively [33].

Similar to our evaluation approach on the DanceTrack dataset, we initially configured SORT-tuned tracker to make the comparison fair. Compared to the defined default tracker parameters, we set the minimum IoU threshold IoU_{min} to 0.05. Detailed information about the used YOLOX object detector can be found in “Appendix SportsMOT”.

Comparison of various state-of-the-art motion-based trackers evaluated on test set is presented in Table 4, while methods that use appearance-based association are included just for reference. We observed that by applying our MoveSORT hybrid association method to the SORT-tuned tracker, we boosted the tracker’s performance by 0.7% in terms of HOTA. Further significant improvements were achieved by replacing the KF with one of our stronger filters. Our Bayesian filters (AR-RNN, ACNP, RNN-CNP, and RNN-ODE) enhanced the SORT performance by up to 3.2% in terms of HOTA, and 2.0% in terms of IDF1. On the other hand, our end-to-end filters (RNNFilter and NODEFilter) served as an even better replacement, yielding up to a 3.6% improvement

Table 4 The evaluation results for various tracking methods on the SportsMOT test set

Method	HOTA	DetA	AssA	MOTA	IDF1
<i>Motion and appearance-based</i>					
FairMOT [56]	49.3	70.2	34.7	86.4	53.5
MixSort-Byte [33]	65.7	78.8	54.8	96.2	74.1
MixSort-OC [33]	74.1	88.5	62.0	96.5	74.4
Deep-ElIoU [57]	77.2	88.2	67.7	96.3	79.8
<i>Motion-based</i>					
CenterTrack [58]	62.7	82.1	48.0	90.8	60.0
ByteTrack [24]	64.1	78.5	52.3	95.9	71.4
BoT-SORT [12]	68.7	84.4	55.9	94.5	70.0
SORT-Tuned	70.3	86	57.5	97.0	71.9
OC_SORT [31]	73.7	88.5	61.5	96.5	74.0
MotionTrack [32]	74.0	88.8	61.7	96.6	74.0
<i>Motion-based</i>					
MoveSORT-KF (ours)	71.0	86.2	58.5	97.1	72.6
MoveSORT-AR-RNN (ours)	74.2	88.8	62.0	97.1	74.6
MoveSORT-ACNP (ours)	73.9	88.8	61.5	97.1	74.4
MoveSORT-RNN-CNP (ours)	74.2	88.8	62.0	97.1	74.6
MoveSORT-RNN-ODE (ours)	73.8	88.8	61.4	97.1	74.1
MoveSORT-RNNFilter (ours)	74.6	87.5	63.7	96.7	76.9
MoveSORT-NODEFilter (ours)	74.5	88.0	63.0	96.8	76.2

The best results for motion-based methods are in bold for each metric, while the second-place results are italicized. Results for trackers that use appearance-based association are included for reference only, as they are not comparable to the other methods. The results for the ByteTrack and OC_SORT trackers are taken from [33]

Table 5 Tracker performance comparison during occlusions on SportsMOT validation set

Filter	Precision	Recall	f1-score
KF	55.1	25.2	34.6
AR-RNN	70.0	30.3	42.3
ACNP	64.6	35.5	45.8
RNN-CNP	63.7	36.6	46.5
RNNODE	62.8	37.8	47.2
RNNFilter	66.3	36.6	47.2
NODEFilter	67.3	32.6	44.0

Numbers are shown as percentages. The best results are in bold for each metric

in terms of HOTA, and a dramatic improvement of 4.3% in terms of IDF1.

We compare our trackers with motion-based variants from the literature on the SportsMOT test set. As indicated in Table 4, MoveSORT-RNNFilter outperforms all motion-based tracking method. Specifically, OC_SORT, the best performing motion-based method on SportsMOT, by 0.9% in terms of HOTA, 0.2% in terms of MOTA, and 2.9% in terms of IDF1. Noteworthy, as of the time of submitting this paper, our MoveSORT-RNN-CNP tracker is ranked first on

the SportsMOT leaderboard in terms of MOTA, surpassing all other trackers.

Occlusion handling analysis. We analyzed tracker performance during occlusions using the same approach employed for the DanceTrack dataset 5.3. The analysis results are presented in Table 5, from which we observe that our motion models outperform the KF. Specifically, the RNNFilter model outperforms the KF by up to 11.2% in terms of precision, 11.4% in terms of recall, and 12.6% in terms of F1-score. These results underscore the limitations of the KF in handling complex occlusions, especially when compared to our proposed motion models.

5.5 Tracker evaluation on MOTChallenge

MOTChallenge⁵ offers two standard datasets for multi-pedestrian tracking—MOT17 and MOT20 [13]. The MOT17 dataset consists of 14 short video scenes (7 in the training set and 7 in the test set) with non-static cameras, and the MOT20 dataset consists of 8 crowded scenes with static cameras with four scenes designated for the training set and the remaining four for the test set. We note that the tracker performance on MOTChallenge primarily benefits from association heuristics and to a lesser extent from motion model improvements (due to the slow-moving objects exhibiting linear motion). Therefore MOT17 and also MOT20 are not proper benchmarks for our filters which are designed for non-linear motion. However, these datasets are considered the standard for tracker evaluation and we include them in our evaluation for transparency.

To ensure a transparent comparison of motion models, we have kept our method simple, adhering to the SORT baseline rather than adopting more complex, heavily engineered variants. We use global tracker hyperparameters and do not fine-tune them based on specific scenes in the validation or test set, arguing that this approach makes the evaluation more realistic. Additionally, to keep the design simple and better distinguish the contributions of our methodology, we do not employ Camera Motion Compensation (CMC) even though it is known to improve performance on non-static camera datasets [12, 34].

Following the MOTChallenge guidelines, we submit only one tracker to the test server. A tracker is chosen based on the results on the publicly available (training) data. In adherence to the standard approach, we divide each training scene into two halves: the first half is used for the training set, and the second half is used for the validation set [12, 24, 34, 49, 53]. Detailed information about the object detector setup can be found in “Appendix MOTChallenge”.

⁵ MOTChallenge (<https://motchallenge.net>) is a platform for evaluating the performance of multiple object tracking algorithms.

Table 6 Comparison of our trackers with the SORT baseline on the MOT17 and MOT20 validation sets

Method	MOT17				MOT20			
	HOTA	MOTA	IDF1	IDSW	HOTA	MOTA	IDF1	IDSW
SORT	68.44	79.65	79.29	202	61.60	80.54	77.51	971
MoveSORT-KF	68.91	79.6	79.74	185	61.7	80.61	77.67	954
MoveSORT-AR-RNN	67.86	79.68	78.29	205	58.78	78.65	72.78	1469
MoveSORT-ACNP	68.09	79.00	78.64	284	61.62	80.68	77.56	962
MoveSORT-RNN-CNP	69.13	79.78	80.52	196	61.55	80.71	77.59	975
MoveSORT-RNN-ODE	68.35	79.28	79.00	227	61.66	80.69	77.66	996
MoveSORT-RNNFilter	68.62	79.41	79.05	199	61.32	80.62	77.05	977
MoveSORT-NODEFilter	67.69	78.61	78.37	250	60.59	80.17	76.07	1102

The best results are in bold for each metric

Table 7 Evaluation results for various state-of-the-art tracking methods on the MOT17 test set

Method	HOTA	DetA	AssA	MOTA	IDF1	IDSW
<i>Motion and appearance-based</i>						
BYTEv2 [50]	63.6	65.0	62.7	80.6	78.9	1239
StrongSORT++ [59]	64.4	64.6	64.4	79.6	79.5	1194
Bot-Sort-ReID ^{BT,*} [12]	65.0	64.9	65.5	80.5	80.2	1212
StrongTBD [60]	65.6	65.9	65.6	81.6	80.8	954
Deep OC_SORT [53]	64.9	64.1	65.9	79.4	80.6	1023
<i>Motion-based</i>						
MotionTrack (MOT17) [32]	60.9	–	59.4	76.5	73.5	–
ByteTrack ^{BT} [24]	63.1	64.5	62.0	80.3	77.3	2196
BYTEv2 ^{BT} [50]	63.6	65.0	62.7	80.6	78.9	1239
OC_SORT [31]	63.2	63.2	63.4	78.0	77.5	1950
MotionTrack ^{BT,*} [34]	65.1	65.4	65.1	81.1	80.1	1140
SparseTrack ^{BT,*} [49]	65.1	65.3	65.1	81.0	80.1	1170
<i>Motion-based</i>						
MoveSORT-RNN-CNP (ours)	62.2	64.3	60.4	79.5	75.1	2688

Methods marked by (*) employ CMC and methods marked by (BT) are extensions of the ByteTrack tracker [24]. Results for trackers that use appearance-based association are included for reference only, as they are not comparable to the other methods. The best results for motion-based methods are in bold for each metric

Method selection. We examine all filter methods on the validation dataset, with detailed insights into the tracker's performance based on the chosen filter architecture, as shown in Table 6. Additionally, to show how well our new association method works, we compare SORT which uses the usual IoU association, and MoveSORT-KF which combines the IoU and L1 association methods. From the results, we can observe that the hybrid association in MoveSORT-KF improves the SORT tracker's performance by reducing frequency of ID switches (IDSW) by 8.5% on MOT17 and 2% on MOT20, and improving HOTA by 0.47% on MOT17 and 0.10% on MOT20. We can also see that all filters exhibit comparable performance. The only exception is RNN-CNP on MO17 which outperforms all other filters. Based on the results presented in Table 6, we have selected MoveSORT-RNN-CNP for evaluation on MOT17 test set and MoveSORT-RNN-ODE for evaluation on MOT20 test set.

Evaluation. We evaluate MoveSORT-RNN-CNP on the MOT17 test set and compare it with current state-of-the-art methods that employ only motion-based association.

These results are presented in Table 7. Our method lags behind state-of-the-art methods employing CMC by 2.9% in HOTA. For methods not employing CMC, which are more comparable with our SORT-based trackers, the gap in HOTA reduces to 1.4%. As expected, trackers which employ advanced heuristics outperform ours which were not designed for linear motion nor apply such heuristics. However, our approach outperforms MotionTrack [32], which is based on the SORT baseline like our method but employs a transformer-based motion model, by 2.3% in HOTA. From the results, we conclude that our tracking method generalizes well even on small datasets.

We evaluate MoveSORT-RNN-ODE on the MOT20 test set and compare it with current state-of-the-art methods that employ only motion-based association. These results are presented in Table 8. In this instance, our method lags behind others by up to 3% in HOTA but has a relatively low number of ID switches, although we did not specifically design it for linear movement. Still, like on MOT17,

Table 8 Evaluation results for various state-of-the-art tracking methods on the MOT20 test set

Method	HOTA	DetA	AssA	MOTA	IDF1	IDSW
<i>Motion and appearance-based</i>						
BYTEv2 [50]	61.4	62.9	60.1	77.3	75.6	1082
StrongSORT++ [59]	62.6	61.3	64.0	73.8	77.0	770
Deep OC_SORT [53]	63.9	62.4	65.7	75.6	79.2	779
Bot-Sort-ReID ^{BT,*} [12]	63.3	64.0	62.9	77.8	77.5	1313
StrongTBD [60]	63.6	64.4	63.0	78.0	77.0	1101
<i>Motion-based</i>						
MotionTrack (MOT20) [32]	58.3	–	52.3	75.0	53.9	–
ByteTrack ^{BT} [24]	61.3	63.4	59.6	77.8	75.2	1223
BYTEv2 ^{BT} [50]	61.4	62.9	60.1	77.3	75.6	1082
OC_SORT [31]	62.4	62.4	62.5	75.7	76.3	942
MotionTrack ^{BT,*} [34]	62.8	64.0	61.8	78.0	76.5	1165
Bot-Sort ^{BT,*} [12]	63.3	64.0	62.9	77.8	77.5	1,313
SparseTrack ^{BT,*} [49]	63.5	64.1	63.1	78.1	77.6	1120
<i>Motion-based</i>						
MoveSORT-RNN-ODE (ours)	60.5	61.3	60.0	74.3	74.0	943

Methods marked by (*) employ CMC and methods marked by (BT) are extensions of the ByteTrack tracker [24]. Results for trackers that use appearance-based association are included for reference only, as they are not comparable to the other methods. The best results for motion-based methods are in bold for each metric

our approach outperforms MotionTrack [32], by 2.2% in HOTA.

5.6 Filter robustness analysis on LaSOT

LaSOT is a popular dataset specifically designed for single object tracking, containing over 1,400 video scenes recorded by non-static cameras that are evenly spread across 70 categories, amounting to more than 3.5 million frames. The extended LaSOT adds 10 new class categories to the test set not found in the initial training and validation sets, increasing the diversity for evaluation and analysis [51, 52].

The dataset reflects the complex behavior of objects that irregularly disappear and reappear due to occlusions or moving out of view. Object tracking on such a dataset is not feasible without appearance features. Since we do not consider appearance features in this work, we do not perform tracker evaluation on this dataset. However, the diversity of motion patterns across numerous categories and scenes renders this dataset an excellent choice for comparing the accuracy of filters' motion models and the efficiency of their filtering approaches. We explore their robustness to detector noise, with a focus on bounding box localization errors and missed detections.

Robustness to Gaussian noise. We evaluate the robustness of different filters by simulating detector bounding box localization error while performing an end-to-end evaluation on 280 scenes (4 for each of 70 categories) from the LaSOT validation dataset. Gaussian noise is added to the ground truth data, with its bounding box covariance matrix being equal to $\text{diag}((\sigma \cdot w)^2, (\sigma \cdot h)^2, (\sigma \cdot w)^2, (\sigma \cdot h)^2)$ where σ varies from 0.0 to 0.3. We operate under the assumption that

the noise statistics are not available for tuning any of the filters. A default measurement noise standard deviation of 0.05 is used as a hyperparameter for all Bayesian filters. We also train a version of AR-RNN, denoted AR-RNN (naive), which was intentionally not subjected to default augmentations during training mentioned in “Appendix [Augmentations](#)”. We refer to the same architecture, but with augmentations applied during training as AR-RNN (robust). This approach was chosen to demonstrate the effectiveness of the applied augmentation methodology.

In Fig. 4 (left), which displays the bounding box filter prediction, it is evident that the AR-RNN (naive) significantly underperforms compared to all other methods when noise is introduced. All filters trained with augmentations surpass the performance of the KF at any noise level, including the case of noiseless detections ($\sigma = 0$). Our end-to-end filters, NODEFilter and RNNFilter, demonstrate the highest robustness to Gaussian noise, as they are specifically designed for noise filtering. This difference becomes even more pronounced for posterior estimations, as seen in Fig. 4 (right). We demonstrate that our end-to-end filters can accurately filter out measurement noise at varying levels, outperforming other methods even when the true noise statistics match those assumed by the Bayesian filters (i.e. when noise standard deviation is $\sigma = 0.05$).

Robustness to missed observations. We also assess the robustness of different filters on the same set of scenes by simulating missed observations (i.e. false negatives). Specifically, we randomly exclude some observations and force the filters to skip the update step. We vary the probability of false negatives from 0 to 0.8. Moreover, we also add Gaussian noise to the ground truth bounding boxes to evaluate the

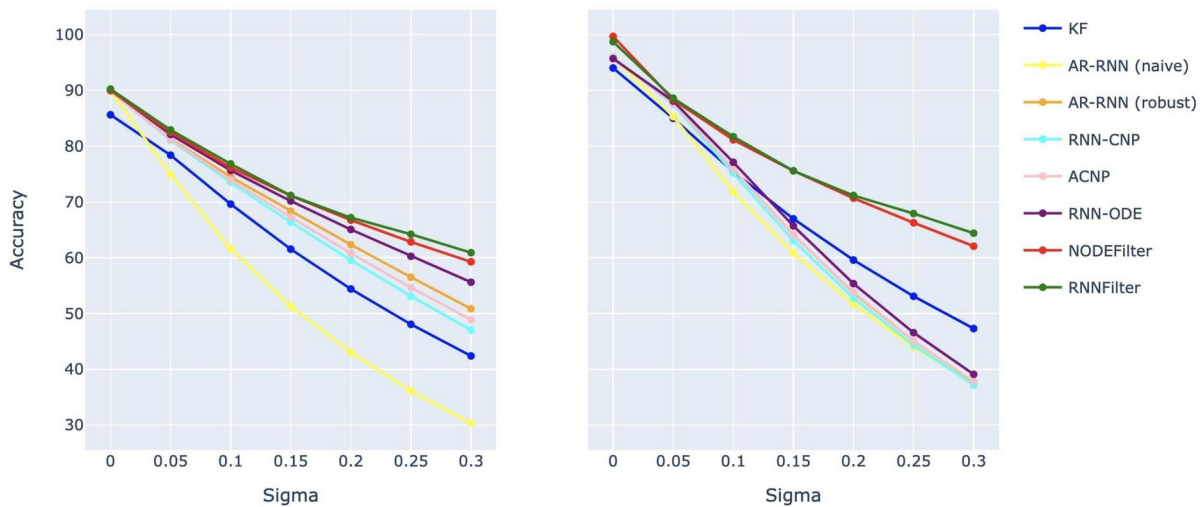


Fig. 4 Evaluation of accuracy for all filters on LaSOT validation dataset for different Gaussian noise standard deviations (left—prior, right—posterior)

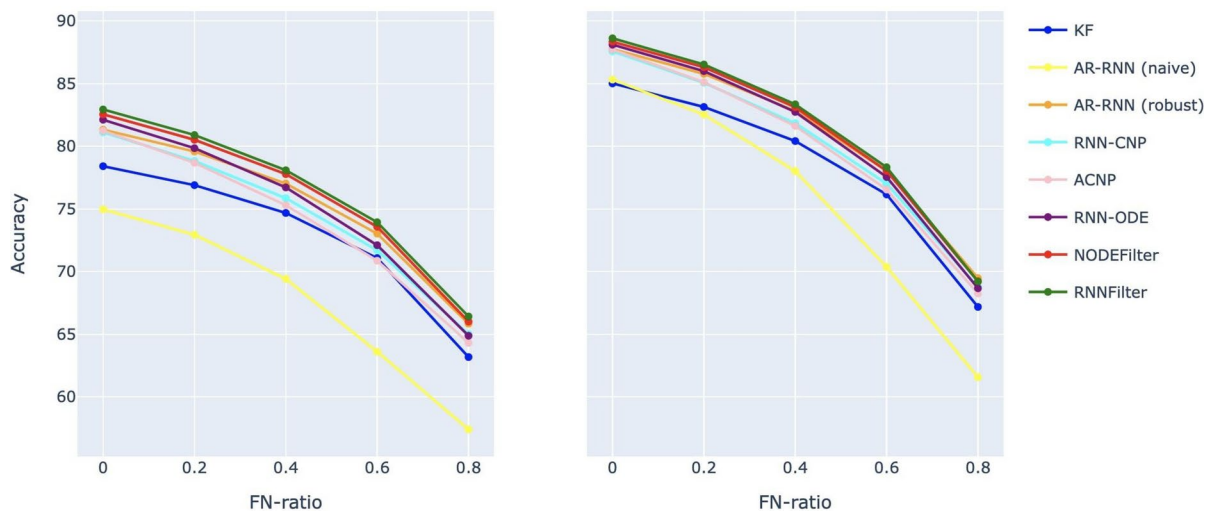


Fig. 5 Evaluation of accuracy on the LaSOT validation dataset for different false negative probabilities (left—prior, right—posterior)

quality of the posterior estimation. The noise standard deviation is fixed at 0.05, aligning with the parameter value used for all Bayesian filters. Note that unlike them the NODEFilter and RNNFilter have to implicitly learn the noise statistics from partly observed trajectories.

In Fig. 5 (left), we observe that the proposed augmentations significantly improve the effectiveness of AR-RNN (robust), which was trained with missing detection augmentations, compared to AR-RNN (naive), which was trained without them. KF lags considerably at different rates of missed detections. The end-to-end filters, NODEFilter and RNNFilter, deliver the strongest results. Figure 5 (right) demonstrates that the effects of missing observations on the posterior estimates are similar to those for the prior. The end-to-end filters are effective even when most of the bounding boxes are not observed.

Robustness to real detector noise. We also assess the robustness of filters when using a real detector. Here, localization errors and missed detections naturally occur as a consequence of the limitations of the real detector, instead of being simulated by adding noise and/or dropping the ground-truth bounding boxes, as was the case up to now. We conduct evaluations on categories where the object detector demonstrates sufficient performance for tracking-by-detection. Detailed information about the object detector setup and category selection for filter evaluation can be found in “Appendix LaSOT”.

In the context of single-object tracking, although multiple objects may be detected in the image, only one is of interest—the one being tracked. For this reason, we must select the most fitting detection bounding box. After performing object detection, only those bounding boxes with a

Table 9 Results on the LaSOT validation set when the observations are produced by a real detector, instead of being synthesized from the ground truth

Model	Prior accuracy	Prior MSE	Posterior accuracy	Posterior MSE
KF	69.03	6.0e−3	73.41	3.0e−3
AR-RNN (naive)	75.65	1.7e−3	79.03	1.6e−3
AR-RNN (robust)	76.02	1.7e−3	79.30	1.5e−3
RNN-CNP	75.87	1.7e−3	78.79	1.5e−3
ACNP	75.66	1.2e−3	79.33	1.5e−3
RNN-ODE	75.32	1.7e−3	79.33	1.5e−3
NODEFilter	75.60	1.7e−3	79.30	1.5e−3
RNNFilter	75.02	2.0e−3	78.58	1.9e−3

The best results are in bold for each metric

Table 10 Speed evaluation for various filters

Model/FPS	Predict	Update	Global
KF	26,147	11,493	7984
AR-RNN	1462	16,637	1397
RNN-CNP	868	21,492	834
ACNP	486	20,487	475
RNN-ODE	307	20,686	303
NODEFilter (rk4)	53	1099	52
NODEFilter (Euler)	129	956	123
RNNFilter	85	1289	80

The best results are in bold for each metric

confidence level higher than 0.5 and an IoU with the ground truth of at least 0.3 are considered. Among these candidates, the one with the highest IoU with the ground truth is chosen as the input for the filter. If no candidates meet these criteria, the detection is considered missed for that frame. Consequently, these filter inputs include both bounding box localization errors and missed detection noise, which are inherent to any real-world detector. We further discuss all the evaluation details in “Appendix [Robustness to detector noise evaluation discussion](#)”.

From Table 9 we observe that all filters perform similarly except for the KF which falls behind for up to 6.99% in prior accuracy and 6.62% in posterior accuracy.

In “Appendix [Evaluation on test set with ground truth](#)”, we present the results of evaluating all filters on the LaSOT test set using the ground truth bounding boxes as observations to simulate the perfect detector. In that evaluation, all data-driven filters outperform the KF by more than 10% in terms of tracking accuracy.

Computational efficiency. We measure the throughput of each method in FPS (frames per second), with a higher FPS indicating faster performance. Results can be seen in Table 10. Neural ODE methods are known to be slower than classic RNN variants, so the RNN-ODE filter is the slowest Bayesian filter and the NODEFilter is the slowest end-to-end filter, as expected. The speed of these filters can be

improved during inference at the cost of accuracy by replacing the ODE solver with a faster one (e.g. Runge–Kutta 4th order method with Euler method). In cases of constrained resources, the KF is easily the best option. Additional computational efficiency analysis related to real-time tracking can be found in “Appendix 3”.

Based on the speed comparison in Table 10 and the evaluated performance on non-linear datasets (i.e., DanceTrack and SportsMOT), we argue that the proposed RNN-CNP Bayesian filter is best suited for use cases with constrained computational resources or in cases in which the detector’s bounding box noise uncertainty can be accurately approximated. In other cases, we suggest that the RNNFilter end-to-end filter offers an excellent balance between robustness, accuracy, and computational efficiency. Additionally, the detector’s bounding box noise uncertainty does not need to be approximated.

6 Conclusion

This study presents two novel filtering approaches designed to enhance object-tracking systems, particularly in non-linear motion scenarios. The first method is a Bayesian filter employing a trainable motion model for predicting an object’s prior position. This prior is then combined with detector measurement using Bayes’ rule to obtain a more precise posterior estimation of the object’s position. The second filtering method, an end-to-end filter, combines the motion model predictions with detector measurements by *learning* to filter errors, minimizing the dependency on domain knowledge and hyperparameter adjustments in the process.

The effectiveness, robustness, and versatility of these filtering methods applied in object tracking have been thoroughly evaluated across several datasets that exhibit different motion properties. The benefits of using the proposed filters are evident in the case of datasets that exhibit non-linear motion patterns. The evaluation included different motion model architectures, demonstrating that the proposed filters achieve significant improvement compared to the Kalman filter. Therefore, we conclude that they can replace the Kalman filter in existing tracking systems and make a strong contribution to the multi-object tracking field. Moreover, our MoveSORT tracker, which integrates these filtering methods, includes an improved association technique that outperforms the traditional SORT method when evaluated on multiple metrics, especially in situations with complex non-linear movement and occlusions.

Our future work, from a tracking perspective, will include the incorporation of visual appearance features, camera motion compensation, and other techniques that we

did not use in this work, having aimed to focus on a proper evaluation of the proposed filters. On the other hand, we are also considering the design of more advanced end-to-end filters based on Neural Processes and their integration with neural ordinary differential equations [21].

Appendix 1: Training probabilistic motion models

This section discusses input features, as described in “Section Bounding box and temporal features”, the training environment detailed in “Appendix Training environment”, and the augmentations outlined in “Appendix Augmentations”.

Bounding box and temporal features

A naive approach to creating input bounding box features is to use absolute coordinates. Since absolute coordinates are not translation invariant, the optimization problem for object’s bounding box trajectory prediction is not trivial. Translation invariance is one of the reasons why convolutional neural networks are effective in computer vision [61]. We propose two simple transformations to convert absolute coordinates into translation invariant ones: *first-order difference* and *relative to last observation*.

Scaled first-order difference is defined as $\mathbf{Y}_i = \frac{\mathbf{X}_{i+1} - \mathbf{X}_i}{t_{i+1} - t_i}$, $i = 1, \dots, n-1$, for trajectory length of n . This can be seen as an approximation of the velocity at step i . These features are translation invariant, and also introduce a useful inductive bias, facilitating faster training. The predictions from the motion model are in the same format as the input features, necessitating the application of an inverse transformation on the model outputs to derive absolute coordinates. The inverse transform is equal to $\mathbf{X}_{n+i} = \mathbf{X}_n + \sum_{j=n+1}^{n+i} \mathbf{Y}_j$, $i = 1, \dots, m$ where m is the predicted trajectory length. Using the *scaled first-order difference* format improves the accuracy of the RNN-CNP by 5.84% when not standardized, and by 7.05% when standardized, as shown in Table 11.

Relative to last observation is defined as $\mathbf{Y}_i = \mathbf{X}_n - \mathbf{X}_i$, $i = 1, \dots, n-1$. In simple terms, the coordinate system is relative to the last observation, which can be viewed as *translation invariant absolute coordinates*. The inverse transform is equal to $\mathbf{X}_{n+i} = \mathbf{X}_n + \mathbf{Y}_{n+i}$, $i = 1, \dots, m$ where m is the predicted trajectory length. Using these features improves the accuracy of RNN-CNP by 6.63% when not standardized and by 6.95% when standardized, as evident in Table 11.

The primary advantage of using *relative to last observation* features, as compared to *scaled first-order differences*,

Table 11 Comparison of accuracies (average overlap) of the RNN-CNP filter with different input features on the DanceTrack validation set using ground truth

Feature type	No modification	Gaussian noise	Missed observations	Multi-step (30)
AC	84.69	61.14	78.60	56.62
SFOD	90.53	57.66	84.15	57.03
RLOC	91.32	67.17	85.32	59.44
SSFOD	91.74	59.79	85.78	59.45
SRLOC	91.64	67.78	85.36	59.50

We evaluate the overlap between the single-step prediction and the ground truth in three scenarios: without any modifications to the ground truth—No modification; with Gaussian noise added to the inputs—Gaussian noise ($\sigma = 0.15$); and with randomly omitted bounding boxes with probability 0.50—Missed observations (FN ratio = 0.5). Additionally, we consider the average overlap for multi-step predictions—Multi-step (30 steps). We use these acronyms for feature types: AC—Absolute Coordinates, SFOD—Scaled First-Order Difference, RLOC—Relative to Last Observation Coordinates, SSFOD—Standardized Scaled First-Order Difference, SRLOC—Standardized Relative to Last Observation Coordinates

lies in their robustness to noise. When the noise is high, velocity approximations can be easily disrupted, such as in cases of sudden direction changes. The *relative to last observation* approach is more robust because its scale, $|\mathbf{X}_n - \mathbf{X}_i|$, is typically much larger than $|\frac{\mathbf{X}_{i+1} - \mathbf{X}_i}{t_{i+1} - t_i}|$, hence harder to disrupt. This is evident in Table 11, where accuracy is compared under conditions of inserted bounding box Gaussian noise.

We can observe that effective trajectory features improve training speed, as demonstrated in Fig. 6. Additionally, performing standardization further enhances training speed.

In addition to the features of the bounding box, we also use time information. However, since absolute time values aren’t very useful, we convert them to values relative to the time point of the last observed bounding box: $T_i = t_i - t_1 + 1$, where t_1 is the time of the oldest observation in the measurement buffer. The final model input features are represented as $\mathbf{Y}_i \parallel T_i$, where $\mathbf{Y}_i$ are the transformed bounding box features, and T_i are the transformed time features.

For the DanceTrack, SportsMOT, and LaSOT datasets, we found the *standardized relative to last observation* feature to be the most effective. Conversely, for the MOT17 and MOT20 datasets, the *standardized scaled first order difference* feature proved to be the best option. We argue that the *standardized scaled first-order differences* are less effective for datasets characterized by non-linear motion due to the accumulation of errors—i.e., the sum of predicted differences—in the inverse transform required to obtain the absolute coordinates. This can be directly seen from the definition of the inverse transformation above.

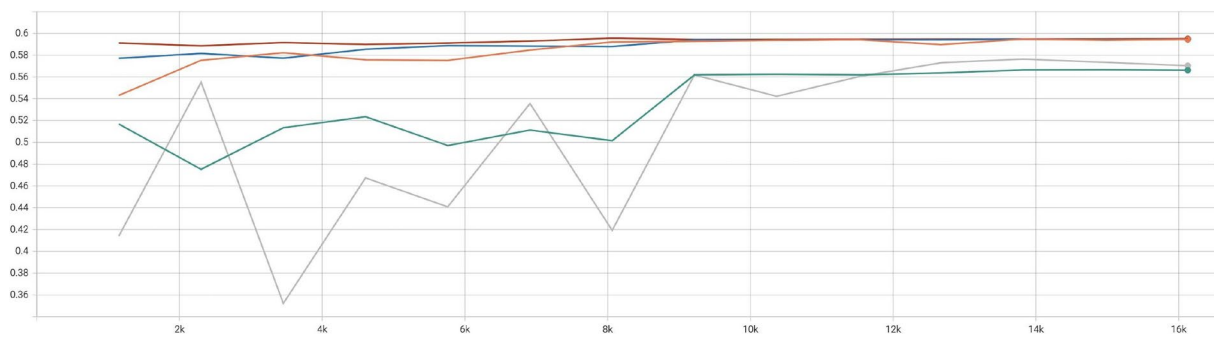


Fig. 6 Comparison of the RNN-CNP motion models trained with different trajectory input features. The x-axis represents the training step, while the y-axis represents accuracy. The graph is best viewed in color: absolute coordinates in gray, scaled first order difference in

green, relative to last observation in orange, standardized scaled first order difference in blue, and standardized relative to last observation in red

Table 12 Comparison of RNN-CNP filter performance with different different augmentations

Augmentations	Single-step	Gaussian noise	Missed observations	Multi-step (30)
None	91.79	53.52	84.10	59.46
Gaussian noise	91.69	68.10	84.54	59.53
Remove points	91.75	54.10	85.23	59.49
Both	91.64	67.78	85.36	59.50

We use the same naming convention as in the Table 11

Training environment

Our default setup includes a batch size of 256 and a learning rate of 0.001, combined with a step scheduler having a multiplier of 0.1 applied each 4 epochs. We employ the AdamW optimizer [62, 63] with weight decay [64]. Note that this configuration may vary between different models. For the ACNP model, we also implement a necessary warm-up, due to the use of the attention mechanism [41], for a period of 3 epochs.

Augmentations

For our default configuration, we employ the following augmentations:

- **Bounding Box Gaussian Noise**—Adds Gaussian noise to the bounding boxes, enhancing the model's robustness to the detector noise. The standard deviation, denoted as *sigma*, is relative to the bounding box height for the y-axis and width for the x-axis. Specifically, for each trajectory, we apply noise with standard deviation value σ of 0.05, 0.10, or 0.25 with probabilities 0.20, 0.05, and 0.01 respectively. Note that with a probability of 0.74, there is no augmentation of this type. For end-to-end filters, we increase these probabilities to 0.60, 0.20,

and 0.05 respectively, as they need to learn to perform noise filtering.

- **Remove Points**—A combination of random bounding box removal, applied with a default probability of 0.2, and trajectory history shortening, i.e. clipping history from the left side, with a probability of 0.05. The trajectory size is always maintained at the minimum of 2 points. These type of augmentations are applied at batch level. In Table 12, we observe that while these augmentations have no significant effect when extrapolation is performed on ground truths, they do make the model more robust to bounding box noise or missed detections.

Appendix 2: Object detection

This section outlines the setup of the object detector for each dataset, including DanceTrack, SportsMOT, MOT17, MOT20, and LaSOT.

DanceTrack

For the DanceTrack evaluation, we use the publicly available DanceTrack YOLOX object detector [5]. This same detector is employed for both the validation and test sets.

SportsMOT

For the SportsMOT test set evaluation, we use the DeepEIoU [57] in conjunction with the YOLOX object detector [5]. This detector was trained using both the training and validation datasets.

Table 13 Comparison of SORT and MoveSORT-RNN-CNP in real-time tracking on the DanceTrack validation set

Tracker	FPS			Tracking performance		
	Tracker	Association	Object detection	HOTA	IDF1	MOTA
SORT	67.4	748.8	74.1	34.7	35.9	66.9
MoveSORT-RNN-CNP	62.5	379.5	74.9	37.5	38.7	67.0

The column *Association* represents the FPS of the association step measured in isolation, which includes the filtering model inference, cost function calculation, and the Hungarian algorithm. The column *Object detection* represents the FPS of the object detector model measured in isolation, and the column *Tracker* represents the combined inference speed of the association and object detection steps. The best results for motion-based methods are in bold for each metric

MOTChallenge

To address the limited data in the MOT17 dataset, the detector intended for the test set evaluation receives additional training on the validation set, in contrast to the detector employed for evaluation on the validation set, in line with the standard practice [12, 24, 53]. For tracker hyperparameter tuning and filter architecture selection on the validation dataset, we use ByteTrack's YOLOX detector, trained on CrowdHuman and the first half of each MOT17 training scene, but not on the MOT17 validation set. The second model, deployed for evaluating the test scenes, is ByteTrack's YOLOX detector trained on both the CrowdHuman and the complete MOT17 datasets.

For evaluation on the MOT20 dataset, we employ ByteTrack's YOLOX detector trained on both CrowdHuman and the full MOT20 training set, which also includes the validation data. This detector cannot be used for tracker hyperparameter tuning and filter architecture selection on the validation dataset, due to the potential for overly optimistic results. Following the approach in [53], we employ the same detector that is used for evaluation on the MOT17 test set.

LaSOT

We fine-tuned YOLOv8 [6] pretrained on the COCO dataset [65] using LaSOT dataset SOT annotations. This results in a detector of lower accuracy, as SOT annotations are not necessarily optimized for multi-object detection training.⁶ For evaluating the detector, we use mAP50 (mean Average Precision for an IoU threshold of 50%), which is a standard metric used in YOLOv8 evaluations [6]. We use only 11 classes that have a mAP50 of more than 0.4, as the noise is too extreme for object tracking with a lower mAP50 score. This set of classes includes categories with mAP50 in parentheses: flag (0.64), tank (0.58), skateboard (0.57), shark (0.41), spider (0.56), airplane (0.54), pool (0.65), tiger (0.46), bicycle (0.57), hippo (0.59), boat (0.66).

⁶ In frames with multiple objects of interest for training a detector, only one is annotated, which is problematic for training an object detector.

Appendix 3: Real-time object tracking performance

We conduct additional experiments to demonstrate that RNN-CNP, which strikes a good balance of speed and predictive quality, can be employed in real-time object tracking without significantly increasing computational requirements. Since the publicly available DanceTrack PyTorch YOLOX-X model, with an input image resolution of 1440×800 , cannot achieve real-time performance (i.e., has less than 10 FPS), we trained a YOLOv8-M model, which is approximately 10 times smaller, with an input image resolution of 640×384 . From Table 13, we observe that MoveSORT-RNN-CNP improves tracking accuracy compared to SORT without a notable increase in computational demands. Thus, we conclude that RNN-CNP achieves good real-time object tracking performance. We note that the tracker inference speed can be even further improved by converting the filter and object detection to a more efficient framework like ONNX [66] or TensorRT [67].

Appendix 4: Additional LaSOT results and discussion

This section contains additional information about the results and analysis on the LaSOT dataset.

Robustness to detector noise evaluation discussion

We explore the reasoning behind our evaluation setup on LaSOT, which involves real detector noise. The KF tends to accumulate errors rapidly in cases of multiple skipped update steps due to missed detections. This error is further magnified on this dataset due to the prevalence of non-linear motion, which is ill-suited to the KF's capabilities. Consequently, the predicted bounding box often drifts outside the boundaries of the frame, leading to a complete loss of tracking accuracy. In contrast, our data-driven filters adeptly learn that an object is usually positioned in the center of the camera view. To make a comparison with the KF more fair,

Table 14 Evaluation of different filters on LaSOT test set in case of ground truth inputs

Model	Prior accuracy
KF	69.85
AR-RNN (robust)	80.66
RNN-CNP	80.17
ACNP	80.39
RNN-ODE	80.13
NODEFilter	80.43
RNNFilter	80.56

we compute the accuracies by excluding those portions of scenes where the detection is missed for 5 or more consecutive frames.

It should be noted that this dataset lacks ground truth annotations during occlusions, meaning that motion models are not evaluated in these conditions. This scenario inadvertently favors the KF and the AR-RNN (naive) filter which are sensitive to missed detections, as can be seen in Fig. 5.

Evaluation on test set with ground truth

We evaluate accuracy of the prediction for each filter with ground truth on the test set with 150 scenes which includes unseen categories. Based on the results in Table 14, we can observe that all non-linear data-driven filters perform similarly. However, they drastically outperform the KF by up to 10.69% in terms of accuracy.

Author contributions M.A. and M.N. conceived the idea. All the authors discussed the idea and the design of the experiments. M.A. carried out the experiments and wrote the first draft. All the authors discussed the results and contributed to the final manuscript.

Data availability All datasets that we performed analysis on are publicly available and can be found following these links: 1. DanceTrack: <https://arxiv.org/abs/2111.14690>, <https://github.com/DanceTrack/DanceTrack>, 2. SportsMOT: <https://arxiv.org/abs/2304.05170>, <https://github.com/MCG-NJU/SportsMOT>, 3. MOT17 and MOT20: <https://motchallenge.net/>, 4. LaSOT: <https://arxiv.org/abs/2009.03465>, https://github.com/HengLan/LaSOT_Evaluation_Toolkit.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Balasubramaniam, A., Pasricha, S.: Object detection in autonomous vehicles: status and open challenges (2022). [arXiv:2201.07706](https://arxiv.org/abs/2201.07706)
- Caesar, H., Bankiti, V., Lang, A.H., Vora, S., Liong, V.E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G., Beijbom, O.: nuscenes: A multimodal dataset for autonomous driving. In: CVPR (2020). [arXiv:1903.11027](https://arxiv.org/abs/1903.11027)
- Xu, Z., Zhan, X., Xiu, Y., Suzuki, C., Shimada, K.: Onboard dynamic-object detection and tracking for autonomous robot navigation with rgb-d camera. In: IEEE Robotics and Automation Letters (2023). [arXiv:2303.00132](https://arxiv.org/abs/2303.00132)
- Urbann, O., Bredtmann, O., Otten, M., Richter, J.-P., Bauer, T., Zibriczky, D.: Online and real-time tracking in a surveillance scenario. In: 5th Workshop on Long-term Human Motion Prediction (2021). [arXiv:2106.01153](https://arxiv.org/abs/2106.01153)
- Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding yolo series in 2021 (2021). [arXiv Preprint arXiv:2107.08430v2](https://arxiv.org/abs/2107.08430)
- Glenn, J., Ayush, C., Jing, Q.: YOLO by Ultralytics (2023). <https://github.com/ultralytics/ultralytics>
- Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: Towards real-time object detection with region proposal networks. In: NIPS (2015). [arXiv:1506.01497](https://arxiv.org/abs/1506.01497)
- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: ECCV (2020). [arXiv:2005.12872](https://arxiv.org/abs/2005.12872)
- Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple online and realtime tracking. In: ICIP (2016). [arXiv:1602.00763](https://arxiv.org/abs/1602.00763)
- Wojke, N., Bewley, A., Paulus, D.: Simple online and real-time tracking with a deep association metric. In: ICIP (2017). [arXiv:1703.07402](https://arxiv.org/abs/1703.07402)
- Wang, Z., Zheng, L., Liu, Y., Li, Y., Wang, S.: Towards real-time multi-object tracking. In: ECCV (2020). [arXiv:1909.12605](https://arxiv.org/abs/1909.12605)
- Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: Bot-sort: Robust associations multi-pedestrian tracking. In: [arXiv Preprint \(2022\). arXiv:2206.14651](https://arxiv.org/abs/2206.14651)
- Dendorfer, P., Rezatofighi, H., Milan, A., Shi, J., Cremers, D., Reid, I., Roth, S., Schindler, K., Leal-Taixe, L.: Mot20: A benchmark for multi object tracking in crowded scenes. In: [arXiv Preprint \(2020\). arXiv:2003.09003](https://arxiv.org/abs/2003.09003)
- Sun, P., Cao, J., Jiang, Y., Yuan, Z., Bai, S., Kitani, K., Luo, P.: Dancetrack: Multi-object tracking in uniform appearance and diverse motion. In: CVPR (2021). [arXiv:2111.14690](https://arxiv.org/abs/2111.14690)
- Gordon, N.J., Salmond, D.J., Smith, A.F.M.: Novel approach to nonlinear/non-gaussian bayesian state estimation. In: International Society for Optics and Photonics (1993)
- Gruber, M.: An approach to target tracking. In: MIT Lexington Lincoln Lab (1967)
- Julier, S.J., Uhlmann, J.K.: New extension of the kalman filter to nonlinear systems. In: International Society for Optics and Photonics (1997)
- Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D.: Neural ordinary differential equations. In: NIPS (2018). [arXiv:1806.07366](https://arxiv.org/abs/1806.07366)
- Garnelo, M., Schwarz, J., Rosenbaum, D., Viola, F., Rezende, D.J., Eslami, S.M.A., Teh, Y.W.: Neural processes. In: ICML (2018). [arXiv:1807.01622](https://arxiv.org/abs/1807.01622)
- Garnelo, M., Rosenbaum, D., Maddison, C.J., Tiago Ramalho, D.S., Shanahan, M., Teh, Y.W., Danilo J. Rezende, S.M.A.E.: Conditional neural processes. In: The Proceedings of Machine Learning Research (2018). [arXiv:1807.01613](https://arxiv.org/abs/1807.01613)
- Norcliffe, A., Bodnar, C., Day, B., Moss, J., Liò, P.: Neural ode processes. In: ICLR (2021). [arXiv:2103.12413](https://arxiv.org/abs/2103.12413)
- Rubanova, Y., Chen, R.T.Q., Duvenaud, D.: Latent odes for irregularly-sampled time series. In: NIPS (2019). [arXiv:1907.03907](https://arxiv.org/abs/1907.03907)
- Yan, H., Du, J., Tan, V.Y.F., Feng, J.: On robustness of neural ordinary differential equations. In: ICLR (2019). [arXiv:1910.05513](https://arxiv.org/abs/1910.05513)

24. Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: Bytetrack: Multi-object tracking by associating every detection box. In: ECCV (2022). [arXiv:2110.06864](https://arxiv.org/abs/2110.06864)
25. Ramshaw, L., E, R., Tarjan: On minimum-cost assignments in unbalanced bipartite graphs. In: HP Laboratories (2012)
26. Revach, G., Shlezinger, N., Ni, X., Escoriza, A.L., Sloun, R.J.G., Eldar, Y.C.: Kalmannet: Neural network aided kalman filtering for partially known dynamics. In: IEEE TSP (2019). [arXiv:2107.10043](https://arxiv.org/abs/2107.10043)
27. Li, S., Zhao, S., Cheng, B., Chen, J.: Noise-Aware Framework for Robust Visual Tracking (2022). <https://doi.org/10.1109/TCYB.2020.2996245>
28. Li, S., Zhao, S., Cheng, B., Zhao, E., Chen, J.: Robust visual tracking via hierarchical particle filter and ensemble deep features. IEEE Trans. Circuits Syst. Video Technol. **30**(1), 179–191 (2020). <https://doi.org/10.1109/TCSVT.2018.2889457>
29. Li, S., Zhao, S., Cheng, B., Chen, J.: Dynamic particle filter framework for robust object tracking. IEEE Trans. Circuits Syst. Video Technol. **32**(6), 3735–3748 (2022). <https://doi.org/10.1109/TCSVT.2021.3109981>
30. Li, S., Zhao, S., Cheng, B., Chen, J.: Part-aware framework for robust object tracking. IEEE Trans. Image Process. **32**, 750–763 (2023). <https://doi.org/10.1109/TIP.2022.3232941>
31. Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-centric sort: Rethinking sort for robust multi-object tracking. In: CVPR (2022). [arXiv:2203.14360](https://arxiv.org/abs/2203.14360)
32. Xiao, C., Cao, Q., Zhong, Y., Lan, L., Zhang, X., Cai, H., Luo, Z., Tao, D.: Motiontrack: Learning motion predictor for multiple object tracking. In: arXiv Preprint (2023). [arXiv:2306.02585](https://arxiv.org/abs/2306.02585)
33. Cui, Y., Zeng, C., Zhao, X., Yang, Y., Wu, G., Wang, L.: Sports-mot: A large multi-object tracking dataset in multiple sports scenes. In: CVPR (2023). [arXiv:2304.05170](https://arxiv.org/abs/2304.05170)
34. Qin, Z., Zhou, S., Wang, L., Duan, J., Hua, G., Tang, W.: Motion-track: Learning robust short-term and long-term motions for multi-object tracking. In: CVPR (2023). [arXiv:2303.10404](https://arxiv.org/abs/2303.10404)
35. Tang, C., Wang, X., Bai, Y., Wu, Z., Zhang, J., Huang, Y.: Learning Spatial-Frequency Transformer for Visual Object Tracking (2023). <https://doi.org/10.1109/TCSVT.2023.3249468>
36. Wang, W., Zhang, X., Ren, S., Sun, J., Wang, J., Wang, Q.: Learning Cross-Attention Discriminators via Alternating Time-Space Transformers for Visual Tracking (2023)
37. Gu, F., Lu, J., Cai, C.: Rpfomer: A robust parallel transformer for visual tracking in complex scenes. IEEE Trans. Instrum. Meas. **71**, 1–14 (2022). <https://doi.org/10.1109/TIM.2022.3170972>
38. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: CVPR (2016). [arXiv:1512.03385](https://arxiv.org/abs/1512.03385)
39. Pontryagin, L.S., Mishchenko, E., Boltyanskii, V., Gamkrelidze, R.: The Mathematical Theory of Optimal Processes (1962)
40. Kim, H., Mnih, A., Schwarz, J., Garnelo, M., Ali Eslami, D.R., Vinyals, O., Teh, Y.W.: Attentive neural processes. In: ICLR (2019). [arXiv:1901.05761](https://arxiv.org/abs/1901.05761)
41. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: NIPS (2017). [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
42. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: NIPS (2019). [arXiv:1912.01703](https://arxiv.org/abs/1912.01703)
43. Nix, D.A., Weigend, A.S.: Estimating the mean and variance of the target probability distribution. In: ICNN (1994)
44. Ba, J.L., Kiros, J.R., Hinton, G.E.: Layer normalization. In: arXiv Preprint (2016). [arXiv:1607.06450](https://arxiv.org/abs/1607.06450)
45. Xu, B., Wang, N., Chen, T., Li, M.: Empirical evaluation of rectified activations in convolution network. In: arXiv Preprint (2015). [arXiv:1505.00853](https://arxiv.org/abs/1505.00853)
46. Merriënboer, K.C., Bahdanau, D., Bengio, Y.: On the properties of neural machine translation: Encoder-decoder approaches (2014). [arXiv:1409.1259](https://arxiv.org/abs/1409.1259)
47. Penny, W.: Bayesian Inference, Dynamical Systems and the Brain. <https://www.fil.ion.ucl.ac.uk/~wpenny/>. Accessed 15 Nov 2023 (2023)
48. Brouwer, E.D., Simm, J., Adam Arany, Y.M.: Gru-ode-bayes: Continuous modeling of sporadically-observed time series. In: NIPS (2019). [arXiv:1905.12374](https://arxiv.org/abs/1905.12374)
49. Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: Sparsetrack: Multi-object tracking by performing scene decomposition based on pseudo-depth. arXiv Preprint (2023). [arXiv:2306.05238](https://arxiv.org/abs/2306.05238)
50. Zhang, Y., Wang, X., Ye, X., Zhang, W., Lu, J., Tan, X., Ding, E., Sun, P., Wang, J.: Bytetrackv2: 2d and 3d multi-object tracking by associating every detection box. In: Combined Ophthalmic Research Rotterdam (2023). [arXiv:2303.15334](https://arxiv.org/abs/2303.15334)
51. Fan, H., Lin, L., Yang, F., Chu, P., Deng, G., Yu, S., Bai, H., Xu, Y., Liao, C., Ling, H.: Lasot: A high-quality benchmark for large-scale single object tracking. In: CVPR (2018). [arXiv:1809.07845](https://arxiv.org/abs/1809.07845)
52. Fan, H., Bai, H., Lin, L., Yang, F., Chu, P., Deng, G., Yu, S., Harshit, M.H., Liu, J., Xu, Y., Liao, C., Yuan, L., Ling, H.: Lasot: A high-quality large-scale single object tracking benchmark. In: IJCV (2020). [arXiv:2009.03465](https://arxiv.org/abs/2009.03465)
53. Maggiolino, G., Ahmad, A., Jinkun Cao, K.K.: Deep oc-sort: Multi-pedestrian tracking by adaptive re-identification. In: ICIP (2023). [arXiv:2302.11813](https://arxiv.org/abs/2302.11813)
54. Runge, C.: Über die Numerische Auflösung Von Differentialgleichungen (1895)
55. Cao, J., Pang, J., Xinshuo Weng, R.K., Kitani, K.: Hota: A higher order metric for evaluating multi-object tracking. In: IJCV (2020). [arXiv:2009.07736](https://arxiv.org/abs/2009.07736)
56. Zhang, Y., Wang, C., Wang, X., Zeng, W., Liu, W.: Fairmot: On the fairness of detection and re-identification in multiple object tracking. Int. J. Comput. Vision **129**(11), 3069–3087 (2021). <https://doi.org/10.1007/s11263-021-01513-4>
57. Huang, H.-W., Yang, C.-Y., Sun, J., Kim, P.-K., Kim, K.-J., Lee, K., Huang, C.-I., Hwang, J.-N.: Iterative scale-up expansioniou and deep features association for multi-object tracking in sports. In: WACV RWS Workshop (2023). [arXiv:2306.13074](https://arxiv.org/abs/2306.13074)
58. Zhou, X., Koltun, V., Krähenbühl, P.: Tracking Objects as Points (2020)
59. Du, Y., Zhao, Z., Song, Y., Zhao, Y., Su, F., Gong, T., Meng, H.: Strongsort: Make deepsort great again. In: CVPR (2022). [arXiv:2202.13514](https://arxiv.org/abs/2202.13514)
60. Stadler, D.: A detailed study of the association task in tracking-by-detection-based multi-person tracking. In: Proceedings of the 2022 Joint Workshop of Fraunhofer IOSB and Institute for Anthropomatics, Vision and Fusion Laboratory. Ed.: J. Beyerer. Karlsruher Schriften zur Anthropomatik / Lehrstuhl für Interaktive Echtzeitsysteme, Karlsruher Institut für Technologie ; Fraunhofer-Inst. für Optonik, Systemtechnik und Bildauswertung IOSB Karlsruhe, vol. 62, pp. 59–85. KIT Scientific Publishing, ??? (2023)
61. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning (2016). <http://www.deeplearningbook.org>

62. P, D., Kingma, Ba, J.: Adam: A method for stochastic optimization. In: ICLR (2014)
63. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: ICLR (2019)
64. Ghosh, A., Behl, H.S., Dupont, E., Torr, P.H.S., Namboodiri, V.: A simple weight decay can improve generalization. In: NIPS (1991)
65. Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P.: Microsoft coco: Common objects in context. In: ECCV (2014)
66. ONNX: Open Neural Network Exchange. <https://onnx.ai/>. Accessed 15 August 2024 (2021)
67. TensorRT: High Performance Deep Learning Inference Library. NVIDIA. Accessed 15 August 2024 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.